

Thiết Kế Cơ Sở Dữ Liệu

Đề Tài: LivingDocs

Nhóm 1

2026

Mục lục

Giới Thiệu Đề Tài	iii
I Thiết Kế Quan Niệm	1
1 Thuật toán 2.1 & Mô hình Conceptual ERD	3
1.1 Phân loại Tác nhân & Phạm vi Nghiệp vụ	3
1.2 Chi tiết các Thực thể Dữ liệu Cốt lõi (Core Data Entities) . .	5
1.3 Mô hình hóa Mối kết hợp & Bản số Dữ liệu	9
2 Sơ đồ Quan niệm ERD	17
2.1 Sơ đồ ERD mức khái niệm	18
II Thiết kế Logic	19
3 Áp dụng Thuật toán 2.2 (Lược đồ Quan hệ & Chuẩn hóa) 3NF	21
III Thiết Kế Vật Lý	39
4 Thiết Kế CSDL Vật Lý & DDL SQL	41
4.1 DDL	41
5 Các Truy vấn Hệ thống Demo	61
5.1 Truy vấn Phân quyền (RBAC): Danh sách Người dùng và Vai trò trong Workspace	61
5.2 Truy vấn Phân quyền: Kiểm tra chi tiết Quyền hạn (Permissions) của một Vai trò	61

5.3	Truy vấn Quản lý Repo: Thống kê số lượng Nhánh (Branch) theo từng Repository	62
5.4	Truy vấn Git Workflow: Chi tiết các Commit nằm trong một Pull Request	62
5.5	Truy vấn Git Workflow: Phân tích Dòng code thay đổi (Code Diff)	63
5.6	Truy vấn Cây cú pháp (AST): Danh sách Thực thể mã nguồn (Code Entities)	63
5.7	Truy vấn Hệ quy AST: Sự phụ thuộc giữa các đoạn code (Dependencies)	63
5.8	Truy vấn Sinh tài liệu: Cấu trúc của một Biểu mẫu (Template)	64
5.9	Truy vấn Automation: Cách Placeholder map với Source Code	64
5.10	Truy vấn Vòng đời Tài liệu: Các Phiên bản của một Tài liệu .	65
5.11	Truy vấn Review: Đọc các Comment góp ý trên Tài liệu . . .	65
5.12	Truy vấn Workflow Phê duyệt: Lịch sử Approval	65
5.13	Truy vấn Grounding: Dẫn chứng mã nguồn của Tài liệu . . .	66
5.14	Truy vấn Cảnh báo Trôi dạt (Drift Alert): Các sai lệch chưa xử lý	66
5.15	Truy vấn Audit: Nhật ký thay đổi hệ thống (Change Log) . .	66
IV	Tổng Kết	69
6	Tổng Kết	71

Giới Thiệu Đề Tài

Tổng quan đề tài

Trong quy trình phát triển phần mềm hiện đại, việc duy trì tính nhất quán giữa tài liệu kỹ thuật (*Documentation*) và mã nguồn thực tế (*Git Codebase*) là một thách thức lớn. Hiện tượng "**Trôi lệch tài liệu**" (*Documentation Drift*) thường xuyên xảy ra khi mã nguồn liên tục được cập nhật nhưng tài liệu không được điều chỉnh kịp thời, dẫn đến lãng phí thời gian bảo trì và gia tăng rủi ro trong vận hành hệ thống.

Hệ thống **LivingDocs** ra đời như một giải pháp toàn diện giúp tự động hóa việc đồng bộ, kiểm duyệt và quản lý tài liệu kỹ thuật theo vết mã nguồn. Hệ thống tích hợp công nghệ AI (*AI Grounding*) để đối soát minh chứng trực tiếp giữa các đoạn mã nguồn với văn bản tài liệu, phát hiện và cảnh báo kịp thời sự trôi lệch dữ liệu.

Phạm vi Kiến trúc Cơ sở Dữ liệu

Nhóm thực hiện đề tài xây dựng và thiết kế kiến trúc Cơ sở dữ liệu (CSDL) cho hệ thống **LivingDocs**, bao phủ 5 phân hệ nghiệp vụ nòng cốt:

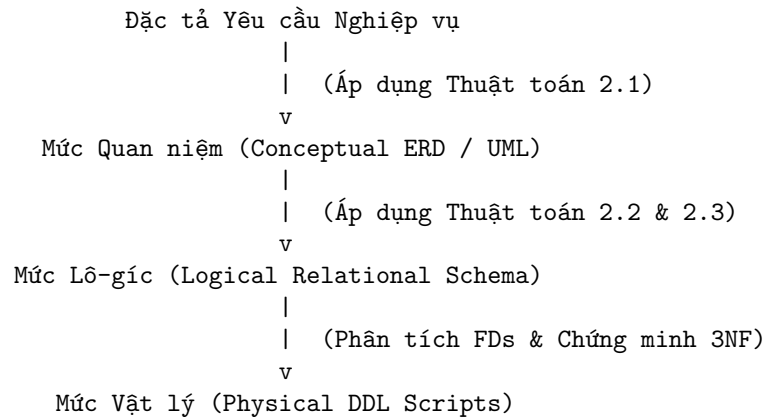
- **Phân hệ Quản trị & Phân quyền (*Auth & Workspace Domain*):** Quản lý thông tin người dùng (*users*), tổ chức không gian làm việc (*workspaces*), cấu trúc vai trò (*roles*) và chi tiết quyền hạn (*permissions*).
- **Phân hệ Tích hợp Mã nguồn (*Git Repository Domain*):** Đồng bộ cấu trúc dữ liệu từ Git bao gồm kho mã nguồn (*repositories*), nhánh (*branches*), lịch sử thay đổi (*commits*), yêu cầu hợp nhất (*pull_requests*) và các thực thể mã nguồn thu được từ cây AST (*code_entities*).
- **Phân hệ Tài liệu & AI Minh chứng (*Documentation & AI Domain*):** Quản lý dàn ý mẫu (*templates*), các mục dàn ý (*template_sections*),

biến chèn (placeholders), tài liệu (documents), các phiên bản tài liệu (document_versions), dữ liệu đối soát AI (grounding_evidences) và hệ thống cảnh báo trôi lệch (drift_alerts).

- **Phân hệ Kiểm duyệt Kỹ thuật (*Code Review Domain*):** Quản lý các phiên review tài liệu (reviews), các quyết định phê duyệt (approvals) và luồng bình luận góp ý (review_comments).
- **Phân hệ Cấu hình & Nhật ký (*Preferences & Audit Logs*):** Quản lý cấu hình thông báo cá nhân (user_notification_preferences), tích hợp kết nối GitHub (github_connections) và lưu trữ nhật ký biến động hệ thống (change_logs).

Phương pháp Nghiên cứu và Quy trình Triển khai CSDL

Để đảm bảo CSDL của hệ thống LivingDocs đạt tính toàn vẹn, tối ưu hiệu năng và triệt tiêu dư thừa dữ liệu, đồ án áp dụng quy trình thiết kế CSDL chuẩn mực qua 3 mức biểu diễn kết hợp triển khai các thuật toán nền tảng của môn học Thiết kế Cơ sở Dữ liệu:



Phần I

Thiết Kế Quan Niệm

Chương 1

Thuật toán 2.1 & Mô hình Conceptual ERD

Ở mức thiết kế quan niệm (*Conceptual Design Level*), mục tiêu cốt lõi là chuyển hóa các mô tả nghiệp vụ thô, yêu cầu chức năng và quy tắc hệ thống thành mô hình dữ liệu trừu tượng, hoàn toàn độc lập với Hệ quản trị cơ sở dữ liệu (DBMS). Đồ án áp dụng **Thuật toán 2.1** (*Algorithm 2.1: Thiết kế Sơ đồ ER từ Yêu cầu Nghiệp vụ*) để đảm bảo tính toàn vẹn và phản ánh chính xác các nghiệp vụ cốt lõi của hệ thống **LivingDocs — Design and Implementation of an AI-Assisted Self-Updating Document System for Engineering Teams**.

Quá trình này tiếp nhận đầu vào (*Input*) là bài toán giải quyết hiện tượng trôi lệch tài liệu (*Documentation Drift*), bộ yêu cầu chức năng từ 5 vai trò người dùng (*Developer, Staff, Manager, Technical Lead, Administrator*) và kết xuất đầu ra (*Output*) là Sơ đồ Quan niệm (*Conceptual ERD/UML*) đã qua kiểm chứng.

1.1 Phân loại Tác nhân & Phạm vi Nghiệp vụ

Tác nhân	Mục tiêu	Nhiệm vụ & Quyền hạn chính	Phạm vi dữ liệu	Đặc điểm cốt lõi
Developer	Khởi tạo và sử dụng tài liệu phần mềm.	Kết nối GitHub và chọn Repository vào Workspace; yêu cầu AI sinh tài liệu theo Template; xem Grounding Evidence; chỉnh sửa bản nháp và Submit; bật/tắt Auto-update; xem Version History và Drift Alert.	Workspace và Repository được cấp quyền; không có quyền Approve hoặc Publish.	Là tác nhân duy nhất khởi tạo vòng đời tài liệu, bắt đầu luồng AI → Staff → Managerr .
Staff	Thực hiện review kỹ thuật cấp 1.	Xem Review Queue; kiểm tra Grounding Evidence, Source File, Code Entity, Commit, Code Diff và Change Log; kiểm tra Template; Comment, sửa lỗi nhỏ; Approve hoặc Reject và trả về Developer.	Toàn bộ bối cảnh kỹ thuật của tài liệu đang review; không có quyền Publish hoặc Commit.	Là cổng lọc kỹ thuật , phát hiện sai sót và AI hallucination trước khi chuyển đến Manager.
Manager	Phê duyệt cuối và quyết định xuất bản tài liệu.	Xem kết quả Staff review, Comment và Grounding Evidence; thực hiện Approve, Reject hoặc Rollback; quản lý danh mục tài liệu cần Manager approval; xem lịch sử Approval.	Tài liệu đã qua Staff review và thuộc phạm vi quản lý.	Là tác nhân duy nhất có quyền Publish hoặc Commit tài liệu vào Repository thật.

Tác nhân	Mục tiêu	Nhiệm vụ & Quyền hạn chính	Phạm vi dữ liệu	Đặc điểm cốt lõi
Technical Lead	Quản trị chất lượng và chính sách tài liệu.	Quản lý Documentation Collection và Ownership; thiết lập chính sách và ngưỡng phát hiện tài liệu lỗi thời ; quản lý Template, Section, Heading, Placeholder và ánh xạ Placeholder ↔ Code Entity.	Toàn bộ Workspace và Repository thuộc phạm vi quản lý.	Tập trung vào Policy, Threshold và Template , không review từng tài liệu như Manager.
Admin	Vận hành hạ tầng và quản trị hệ thống.	Quản lý Tài khoản & Phân quyền: User, Role, Permission, Workspace và Repository và cấu hình tích hợp & hạ tầng như GitHub App, OAuth, GitHub Actions, tích hợp GitLab, Jira, Slack; cấu hình ngôn ngữ và AI Model.	Toàn bộ hệ thống ở cấp độ hạ tầng và kỹ thuật; không review hoặc approve tài liệu.	Là tác nhân duy nhất quản lý cấu hình AI Model như LLM và Embedding, nằm ngoài luồng phê duyệt tài liệu.

1.2 Chi tiết các Thực thể Dữ liệu Cốt lõi (Core Data Entities)

Tên thực thể	Loại thực thể	Mô tả	Thuộc tính
User	Thực thể mạnh	Người dùng cuối trong hệ thống (Developer, Staff, Manager, Tech Lead, Admin).	PK: user_id; email; password_hash; FullName (Phức hợp: FirstName, LastName); NotificationPreferences (đa trị); status; created_at; updated_at; deleted_at
Workspace	Thực thể mạnh	Không gian làm việc chứa repository và tài liệu của một nhóm/dự án.	PK: workspace_id; name; description; status; created_at; updated_at; deleted_at
Role	Thực thể mạnh	Vai trò của người dùng, được scoped theo Workspace.	PK: role_id; role_name; description; created_at; updated_at
Permission	Thực thể mạnh	Quyền hạn chi tiết trên hệ thống, ví dụ DOC_APPROVE, TEMPLATE_EDIT.	PK: permission_id; permission_code; description; created_at
GitHubConnection	Thực thể phụ thuộc User	Lưu thông tin xác thực OAuth của User với GitHub.	PK: connection_id; github_user_id; github_username; access_token; scopes (đa trị); created_at; updated_at
Repository	Thực thể mạnh	Kho chứa mã nguồn được đồng bộ vào hệ thống.	PK: repository_id; name; url; default_branch; sync_status; last_synced_at; created_at; updated_at
Branch	Thực thể yếu – phụ thuộc Repository	Nhánh phát triển trong repository.	PK: branch_id; branch_name; is_default; created_at; updated_at

Tên thực thể	Loại thực thể	Mô tả	Thuộc tính
Commit	Thực thể yếu – phụ thuộc Repository	Bản ghi một lần commit mã nguồn.	PK: commit_id; AuthorInfo (AuthorName, AuthorEmail); message; committed_at; created_at
PullRequest	Thực thể yếu – phụ thuộc Repository	Yêu cầu hợp nhất nhánh (PR) từ GitHub.	PK: pr_id; pr_number; title; description; source_branch; target_branch; status; labels (đa trị); created_at; updated_at; merged_at
CodeDiff	Thực thể yếu – phụ thuộc Commit/PR	Chi tiết thay đổi mã nguồn trong commit hoặc PR.	PK: diff_id; file_path; change_type; additions; deletions; diff_content; created_at
File	Thực thể yếu – phụ thuộc Repository	Cấu trúc tập tin mã nguồn trong kho chứa.	PK: file_id; file_path; file_type; status; created_at; updated_at
CodeEntity	Thực thể yếu – phụ thuộc File	Đơn vị mã nguồn cụ thể thu được từ AST (Class, Method, Function...).	PK: entity_id; entity_name; entity_type; signature; doc_comment; position (start_line, end_line); created_at; updated_at
Template	Thực thể mạnh	Khuôn mẫu chuẩn để AI/User sinh tài liệu.	PK: template_id; name; description; content_structure; creator_type; status; created_at; updated_at
TemplateSection	Thực thể yếu – phụ thuộc Template	Mục con/heading trong cấu trúc dàn ý của Template.	PK: section_id; heading_title; order_index; prompt_instruction; created_at; updated_at

Tên thực thể	Loại thực thể	Mô tả	Thuộc tính
Placeholder	Thực thể yếu – phụ thuộc TemplateSection	Biến/điểm chèn trong Section, nhận dữ liệu từ code.	PK: placeholder_id; variable_name; expected_data_type; description; is_required; created_at; updated_at
Document	Thực thể mạnh	Tài liệu gốc như README, API Reference, Architecture Guide, ADR...	PK: document_id; title; document_type; status; created_at; updated_at; deleted_at
DocumentVersion	Thực thể yếu – phụ thuộc Document	Bản ghi nội dung tài liệu theo từng mốc thời gian và trạng thái (Draft, In Review, Approved, Published).	PK: version_id; version_number; content; change_summary; status; ticket_codes; created_at; updated_at
ChangeLog	Thực thể yếu – phụ thuộc DocumentVersion	Lịch sử vết thay đổi chi tiết giữa các phiên bản tài liệu.	PK: log_id; change_type; change_description; diff_content; created_at; updated_at
Review	Thực thể yếu – phụ thuộc DocumentVersion	Tiến trình Staff thực hiện đánh giá kỹ thuật trên một DocumentVersion.	PK: review_id; status; decision; summary_notes; created_at; updated_at
ReviewComment	Thực thể yếu – phụ thuộc Review	Phản hồi, góp ý chi tiết trong quá trình kiểm duyệt.	PK: comment_id; content; is_resolved; created_at; updated_at
GroundingEvidence	Thực thể yếu – phụ thuộc DocumentVersion	Bảng minh chứng chứa liên kết nguồn gốc (File, CodeEntity, Commit) AI dùng để tạo nội dung, phục vụ chống hallucination.	PK: evidence_id; snippet_content; purpose; created_at

Tên thực thể	Loại thực thể	Mô tả	Thuộc tính
DriftAlert	Thực thể sự kiện/thực thể yếu	Cảnh báo phát hiện tài liệu bị lệch ngữ nghĩa/lỗi thời so với code mới.	PK: alert_id; drift_type; status; detected_at
Approval	Thực thể mối kết hợp / Sự kiện	Quyết định phê duyệt xuất bản hoặc commit tài liệu chính thức từ phía Manager.	PK: approval_id; FK: version_id; FK: user_id; decision; rejection_reason; approved_at

1.3 Mô hình hóa Mối kết hợp & Bản số Dữ liệu

Mối quan hệ & Tỷ lệ bản số	Thuộc tính mối quan hệ	Mô tả nghiệp vụ
A. Nhóm Quản trị & Kết nối (Auth & Workspace Domain)		
User — Workspace — Role <participates> <i>Loại:</i> 3 ngôi (Mạnh - Mạnh - Mạnh) <i>Tỷ lệ:</i> M : N : 1	joined_at, status	Ghi nhận sự tham gia của User vào Workspace. Mỗi User trong 1 Workspace bắt buộc được gán đúng 1 Role xác định (Scoped RBAC). Một User có thể tham gia nhiều Workspace với vai trò độc lập.
Role — contains — Permission <contains> <i>Loại:</i> Role (Mạnh) ↔ Permission (Mạnh) <i>Tỷ lệ:</i> M : N	granted_at	Một Role chứa nhiều Permission; một Permission có thể thuộc nhiều Role khác nhau.
User — connects — Github_Connection <i>Loại:</i> User (Mạnh) ↔ Github_Connection (Phụ thuộc User) <i>Tỷ lệ:</i> 1 : 1	Không có	User ủy quyền và liên kết tài khoản cá nhân với GitHub qua OAuth2. Lưu access_token và scopes phục vụ đồng bộ repository, commit, Pull Request, webhook.
B. Nhóm Mã nguồn & Cấu trúc Code (VCS & Code Domain)		
Workspace — owns — Repository <i>Loại:</i> Workspace (Mạnh) ↔ Repository (Mạnh) <i>Tỷ lệ:</i> 1 : N	Không có (workspace_id FK NOT NULL)	Một Workspace quản lý nhiều Repository; mỗi Repository thuộc quyền sở hữu của đúng một Workspace.

Mối quan hệ & Tỷ lệ bản số	Thuộc tính mối quan hệ	Mô tả nghiệp vụ
Github_Connection — syncs — Repository <i>Loại:</i> Github_Connection (Phụ thuộc User) ↔ Repository (Mạnh) Tỷ lệ: 1 : N	Không có (github_connection_id FK Nullable)	Một kết nối GitHub có thể dùng để đồng bộ nhiều Repository trong Workspace; một Repository gắn với tối đa 1 kết nối đang hoạt động (có thể rỗng nếu bị thu hồi).
Repository — has_branch — Branch <i>Loại:</i> Repository (Mạnh) ↔ Branch (Yếu – phụ thuộc Repository) Tỷ lệ: 1 : N	Không có (repository_id FK NOT NULL)	Một Repository có ít nhất một nhánh mặc định; mỗi Branch bắt buộc thuộc đúng một Repository.
Repository — contains_commit — Commit <i>Loại:</i> Repository (Mạnh) ↔ Commit (Yếu – phụ thuộc Repository) Tỷ lệ: 1 : N	Không có (repository_id FK NOT NULL)	Đảm bảo mọi commit (kể cả commit mồ côi không còn trên branch nào) vẫn xác định được thuộc Repository nào.
Repository — receives_pr — PullRequest <i>Loại:</i> Repository (Mạnh) ↔ PullRequest (Yếu – phụ thuộc Repository) Tỷ lệ: 1 : N	Không có (repository_id FK NOT NULL)	Một Repository chứa nhiều Pull Request; mỗi PR gửi về đúng một Repository.
Branch — includes — Commit <i>Loại:</i> Branch (Yếu) ↔ Commit (Yếu) Tỷ lệ: N : N	Bảng trung gian branch_commit	Một Branch tham chiếu tới nhiều Commit; một Commit có thể được nhiều Branch cùng chứa hoặc không thuộc branch nào (dangling commit).
Commit — has_parent — Commit <i>Loại:</i> Commit (Yếu) ↔ Commit (Yếu) — Tự quan hệ Tỷ lệ: N : N (child ↔ parent)	Bảng trung gian commit_parent	Một commit có thể có 0 cha (gốc), 1 cha (thường) hoặc nhiều cha (merge commit); một commit có thể là cha của nhiều commit con.

Mối quan hệ & Tỷ lệ bản số	Thuộc tính mối quan hệ	Mô tả nghiệp vụ
PullRequest — references — Branch <i>Loại:</i> PullRequest (Yếu) ↔ Branch (Yếu) — 2 vai trò (source / target) Tỷ lệ: $2 \times (1 : N)$	source_branch_id , target_branch_id (FKs, Nullable)	Mỗi PR tham chiếu một nhánh nguồn (source) và một nhánh đích (target); một Branch có thể là nguồn hoặc đích của nhiều PR khác nhau.
PullRequest — includes — Commit <i>Loại:</i> PullRequest (Yếu) ↔ Commit (Yếu) Tỷ lệ: $N : N$	Bảng trung gian pull_request_commit	Một PR tổng hợp một hoặc nhiều commit; một commit có thể xuất hiện trong nhiều PR khác nhau.
User — authors / merges — PullRequest <i>Loại:</i> User (Mạnh) ↔ PullRequest (Yếu) — 2 vai trò (author / merged_by) Tỷ lệ: $2 \times (1 : N)$	author_user_id (FK, NOT NULL), merged_by_user_id (FK, Nullable)	Mỗi PR bắt buộc có đúng một tác giả; PR chỉ có người merge sau khi đã được hoàn tất phê duyệt và merge vào nhánh chính.
User — commits <i>Loại:</i> Không dùng FK toàn vẹn CSDL Tỷ lệ: Trích xuất Git metadata	author_name , author_email	Thông tin tác giả commit lưu trực tiếp dạng Text từ Git metadata; việc đối chiếu với User đăng ký hệ thống là xử lý nghiệp vụ riêng.
Commit — generates_diff — CodeDiff <i>Loại:</i> Commit (Yếu) ↔ CodeDiff (Yếu – phụ thuộc Commit) Tỷ lệ: $1 : N$	Không có	Một Commit sinh ra một hoặc nhiều bản ghi CodeDiff; mỗi CodeDiff thuộc về duy nhất một Commit.
Commit — modifies — File <modifies> <i>Loại:</i> Commit (Yếu) ↔ File (Yếu – phụ thuộc Repository) Tỷ lệ: $M : N$	change_type , lines_added , lines_deleted	Một Commit chỉnh sửa ít nhất một File; một File có thể được sửa đổi qua nhiều Commit (hoặc chưa từng bị sửa).

Mối quan hệ & Tỷ lệ bản số	Thuộc tính mối quan hệ	Mô tả nghiệp vụ
Repository — contains_file — File <i>Loại:</i> Repository (Mạnh) ↔ File (Yếu – phụ thuộc Repository) Tỷ lệ: 1 : N	Không có	Một Repository quản lý cây thư mục chứa nhiều File; mỗi File thuộc một Repository duy nhất.
File — parses_into — CodeEntity <i>Loại:</i> File (Yếu) ↔ CodeEntity (Yếu – phụ thuộc File) Tỷ lệ: 1 : N	Không có	Một File qua bóc tách AST thu được nhiều CodeEntity (Class, Function...); mỗi CodeEntity nằm trọn trong đúng một File.
CodeEntity — depends_on — CodeEntity <depends_on> <i>Loại:</i> CodeEntity (Yếu) ↔ CodeEntity (Yếu) — Tự quan hệ Tỷ lệ: M : N	dependency_type (import, function_call, inherits)	Xác định quan hệ phụ thuộc (gọi hàm, kế thừa, import) giữa các CodeEntity trong hệ thống.
C. Nhóm Khuôn mẫu Tài liệu (Template Engine Domain)		
Workspace — defines — Template <i>Loại:</i> Workspace (Mạnh) ↔ Template (Mạnh) Tỷ lệ: 1 : N	Không có	Một Workspace định nghĩa nhiều Template; mỗi Template thuộc sở hữu của một Workspace.
Template — has_section — TemplateSection <i>Loại:</i> Template (Mạnh) ↔ TemplateSection (Yếu – phụ thuộc Template) Tỷ lệ: 1 : N	Không có	Một Template cấu thành từ ít nhất một Section; mỗi Section nằm trên đúng một Template.
TemplateSection — contains_placeholder — Placeholder <i>Loại:</i> TemplateSection (Yếu) ↔ Placeholder (Yếu – phụ thuộc Section) Tỷ lệ: 1 : N	Không có	Mỗi Section chứa nhiều Placeholder (biến chèn tự động). Khi xoá Section, toàn bộ Placeholder trong đó bị xoá theo (cascade).

Mối quan hệ & Tỷ lệ bản số	Thuộc tính mối quan hệ	Mô tả nghiệp vụ
Placeholder — maps_to — CodeEntity <i>Loại:</i> Placeholder (Yếu) ↔ CodeEntity (Yếu) qua TemplateMapping Tỷ lệ: M : N	extraction_rule , is_active (trên TemplateMapping)	Mỗi bản ghi TemplateMapping chỉ định cho AI biết lấy thông tin từ CodeEntity nào để điền vào Placeholder tương ứng trong Template .
D. Nhóm Tài liệu & Vòng đời Dữ liệu (Documentation Domain)		
Workspace — manages — Document <i>Loại:</i> Workspace (Mạnh) ↔ Document (Mạnh) Tỷ lệ: 1 : N	Không có	Workspace quản lý danh mục Document gốc của dự án; mỗi Document thuộc đúng một Workspace .
User — authors — Document <i>Loại:</i> User (Mạnh) ↔ Document (Mạnh) Tỷ lệ: 1 : N	Không có	User đứng tên tác giả khởi tạo Document ; mỗi Document có một tác giả duy nhất.
Document — has_version — DocumentVersion <i>Loại:</i> Document (Mạnh) ↔ DocumentVersion (Yếu – phụ thuộc Document) Tỷ lệ: 1 : N	Không có	Một Document chứa ít nhất một DocumentVersion ; mỗi phiên bản đại diện cho một mốc chỉnh sửa của Document gốc.
DocumentVersion — tracks_change — ChangeLog <i>Loại:</i> DocumentVersion (Yếu) ↔ ChangeLog (Yếu – phụ thuộc Version) Tỷ lệ: 1 : N	Không có	Mỗi DocumentVersion có thể ghi nhận nhiều lượt ChangeLog chi tiết.
E. Nhóm Kiểm duyệt & Phê duyệt (Review & Governance Domain)		

Mối quan hệ & Tỷ lệ bản số	Thuộc tính mối quan hệ	Mô tả nghiệp vụ
DocumentVersion — has_review — Review <i>Loại:</i> DocumentVersion (Yếu) ↔ Review (Yếu – phụ thuộc Version) Tỷ lệ: 1 : N	Thuộc tính thực thể Review: status, decision, summary_notes, created_at	Một DocumentVersion có thể nhận 0 hoặc nhiều lượt Review; mỗi Review bắt buộc thuộc về đúng một DocumentVersion.
User — performs_review — Review <i>Loại:</i> User (Mạnh) ↔ Review (Yếu) Tỷ lệ: 1 : N	Không có	Một User có thể thực hiện 0 hoặc nhiều Review; mỗi Review do đúng một User chịu trách nhiệm.
Review — contains_comment — ReviewComment <i>Loại:</i> Review (Sự kiện) ↔ ReviewComment (Yếu – phụ thuộc Review) Tỷ lệ: 1 : N	Không có	Một Review chứa nhiều ReviewComment; mỗi comment gắn chặt với một Review.
DocumentVersion — requires_approval — Approval <i>Loại:</i> DocumentVersion (Yếu) ↔ Approval (Yếu – phụ thuộc Version) Tỷ lệ: 1 : N	Thuộc tính thực thể Approval: decision, rejection_reason, approved_at	Một DocumentVersion có thể có 0 hoặc nhiều lượt Approval; mỗi Approval bắt buộc thuộc đúng một DocumentVersion.
User — grants_approval — Approval <i>Loại:</i> User (Mạnh) ↔ Approval (Yếu) Tỷ lệ: 1 : N	Không có	Một User (Manager) có thể thực hiện 0 hoặc nhiều Approval; mỗi bản ghi Approval do đúng một User ra quyết định.
F. Nhóm AI Evidence & Giám sát (Observability Domain)		

Mối quan hệ & Tỷ lệ bản số	Thuộc tính mối quan hệ	Mô tả nghiệp vụ
DocumentVersion — has_evidence — GroundingEvidence <i>Loại:</i> DocumentVersion (Yếu) ↔ GroundingEvidence (Yếu) Tỷ lệ: 1 : N	Không có	DocumentVersion lưu trữ bằng minh chứng đối soát nguồn gốc code để chống hallucination của AI.
GroundingEvidence — references_code — CodeEntity <references_code> <i>Loại:</i> GroundingEvidence (Yếu) ↔ CodeEntity (Yếu) Tỷ lệ: M : N	similarity_score , match_type	Minh chứng AI liên kết trực tiếp với các CodeEntity cụ thể để phục vụ đối soát sai lệch.
GroundingEvidence — detects — DriftAlert <i>Loại:</i> GroundingEvidence (Yếu) ↔ DriftAlert (Sự kiện) Tỷ lệ: 1 : N	Không có	Khi GroundingEvidence phát hiện bất đồng giữa tài liệu và mã nguồn mới, hệ thống kích hoạt một DriftAlert.

Chương 2

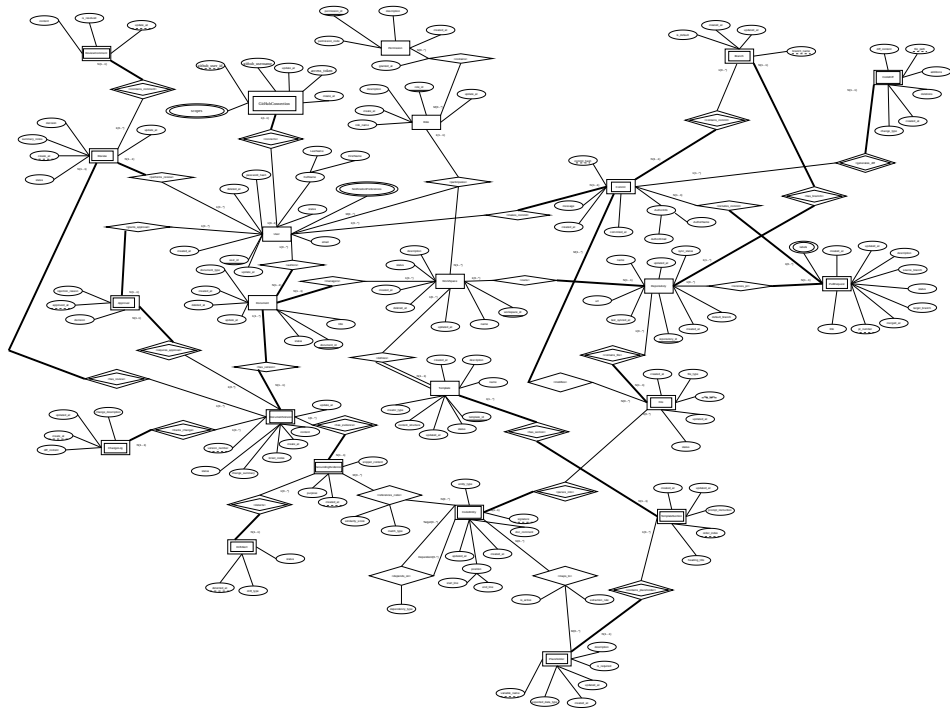
Sơ đồ Quan niệm ERD

Đầu ra của Stage 1 (Thiết kế quan niệm) bao gồm sơ đồ ERD và UML Class Diagram, biểu diễn trù tượng toàn bộ các thực thể, thuộc tính và mối kết hợp nghiệp vụ của hệ thống LivingDocs ở mức logic cao nhất, hoàn toàn độc lập với bất kỳ hệ quản trị CSDL vật lý nào.

Mô hình đã xác định chính xác các bản số ($1 : 1$, $1 : N$, $M : N$) và quy tắc ràng buộc miền, bao phủ trọn vẹn từ quản trị không gian làm việc, cấu trúc kho code cho đến luồng kiểm duyệt 2 cấp ($\text{Staff} \rightarrow \text{Manager}$) và cơ chế lưu minh chứng AI chống ảo giác.

Đây là bức tranh kiến trúc dữ liệu hoàn chỉnh, đóng vai trò là đầu vào (input) chuẩn xác để thực hiện ánh xạ sang Lược đồ quan hệ (Logical Schema) và chuẩn hóa 3NF ở Stage 2.

2.1 Sơ đồ ERD mức khái niệm



Hình 2.1: Sơ đồ ERD mức khái niệm

Phần II

Thiết kế Logic

Chương 3

Áp dụng Thuật toán 2.2 (Lược đồ Quan hệ & Chuẩn hóa) 3NF

Ở mức thiết kế logic (*Logical Design Level*), mục tiêu cốt lõi là chuyển hóa mô hình dữ liệu quan niệm trừu tượng (ERD/UML) thành lược đồ quan hệ lô-gíc (*Logical Relational Schema*) cụ thể, xác định rõ cấu trúc các bảng, cột, khóa chính (PK), khóa ngoại (FK) và các ràng buộc toàn vẹn [1, 2]. Đồ án áp dụng **Thuật toán 2.2** (*Algorithm 2.2: Ánh xạ ERD sang Lược đồ Quan hệ*) để đảm bảo tính toàn vẹn tham chiếu, loại bỏ các dị thường dữ liệu và đưa toàn bộ lược đồ đạt tiêu chuẩn chuẩn hóa 3NF cho hệ thống **LivingDocs — Design and Implementation of an AI-Assisted Self-Updating Document System for Engineering Teams** [3-5].

Quá trình này tiếp nhận đầu vào (*Input*) là Sơ đồ Quan niệm (*Conceptual ERD/UML*) đã qua kiểm chứng ở Stage 1, và kết xuất đầu ra (*Output*) là Lược đồ Quan hệ Lô-gíc (*Logical Schema*) hoàn chỉnh, hồ sơ Ràng buộc Khóa ngoại/Giá trị miền (*FK & CHECK Rules*) và Báo cáo chứng minh Chuẩn hóa 3NF [3, 5-12].

Quy trình Thuật toán 2.2 (*Relational Mapping Algorithm*):

- **Đầu vào (*Input*):** Sơ đồ Quan niệm ERD/UML và từ điển dữ liệu hoàn chỉnh từ Stage 1.
- **Bước 1 (Thực thể mạnh):** Ánh xạ các thực thể mạnh thành các bảng chính, chuyển thuộc tính đơn thành cột và thiết lập Khóa chính (PK).

- **Bước 2 (Thực thể yếu):** Ánh xạ các thực thể yếu thành các bảng phụ thuộc, gán Surrogate PK kiểu UUID và cài đặt FK trở về thực thể chủ.
- **Bước 3 & 4 (Mối kết hợp 1 : 1 và 1 : N):** Chèn Khóa ngoại (FK) từ bảng phía 1 sang bảng phía N (hoặc phía tham gia bắt buộc) kèm các ràng buộc NOT NULL/UNIQUE phù hợp.
- **Bước 5 (Mối kết hợp $M : N$):** Khởi tạo các bảng trung gian (*Junction Tables*) chứa cặp Khóa ngoại (FK) trở về hai bảng gốc và các thuộc tính riêng của mối kết hợp.
- **Bước 6 (Thuộc tính đặc biệt):** Phẳng hóa các thuộc tính phức hợp thành các cột đơn độc lập và tách các thuộc tính đa trị thành các bảng phụ riêng.
- **Bước 7 (Ràng buộc & Chuẩn hóa):** Khai báo tập Phụ thuộc hàm (FDs), chứng minh lược đồ đạt chuẩn 3NF và thiết lập các ràng buộc toàn vẹn (*FK Constraints*).
- **Đầu ra (*Output*):** Lược đồ Quan hệ Lô-gíc (*Logical Schema*) hoàn chỉnh đạt chuẩn 3NF và hồ sơ ràng buộc toàn vẹn.

1. Chuyển đổi Thực thể mạnh thành Bảng chính (Bước 1)

Bảng 3.1: Chuyển đổi Thực thể mạnh thành Bảng chính

STT	Thực thể (ERD)	Bảng CSDL (snake_case)	Ghi chú thiết kế
1	User	users	Khóa chính (PK): user_id (UUID)
2	Workspace	workspaces	Khóa chính (PK): workspace_id (UUID)
3	Role	roles	Khóa chính (PK): role_id (UUID)
4	Permission	permissions	Khóa chính (PK): permission_id (UUID)

Bảng 3.1: Chuyển đổi Thực thể mạnh thành Bảng chính

STT	Thực thể (ERD)	Bảng CSDL (snake_case)	Ghi chú thiết kế
5	Repository	repositories	Khóa chính (PK): repository_id (UUID)
6	Template	templates	Khóa chính (PK): template_id (UUID)
7	Document	documents	Khóa chính (PK): document_id (UUID)

2. Chuyển đổi Thực thể yếu thành Bảng phụ (Bước 2)

Bảng 3.2: Chuyển đổi Thực thể yếu thành Bảng phụ

STT	Bảng phụ (Thực thể yếu)	Bảng cha trực tiếp	Khóa ngoại (FK) & Ràng buộc bổ sung
1	github_connections	users	user_id (FK, UNIQUE)
2	branches	repositories	repository_id (FK)
3	commits	repositories	repository_id (FK) — phụ thuộc repositories; Branch-Commit và PullRequest-Commit là N:N qua bảng trung gian; không có FK tới User
4	pull_requests	repositories	repository_id (FK)
5	code_diffs	commits	commit_id (FK)
6	files	repositories	repository_id (FK)
7	code_entities	files	file_id (FK)
8	template_sections	templates	template_id (FK)
9	placeholders	template_sections	section_id (FK)
10	document_versions	documents	document_id (FK)
11	change_logs	document_versions	version_id (FK)
12	reviews	document_versions	version_id (FK); FK bổ sung: reviewer_user_id → users
13	review_comments	reviews	review_id (FK)

Bảng 3.2: Chuyển đổi Thực thể yếu thành Bảng phụ

STT	Bảng phụ (Thực thể yếu)	Bảng cha trực tiếp	Khóa ngoại (FK) & Ràng buộc bổ sung
14	approvals	document_versions	version_id (FK); FK bổ sung: approver_user_id → users
15	grounding_evidences	document_versions	version_id (FK)
16	drift_alerts	grounding_evidences	evidence_id (FK)

3. Chuyển đổi Mỗi kết hợp M:N thành Bảng trung gian (Bước 5)

Bảng 3.3: Chuyển đổi Mỗi kết hợp M:N thành Bảng trung gian

STT	Bảng trung gian	Mỗi kết hợp gốc	Danh sách các Khóa ngoại (FK)	Ghi chú
1	workspace_members	<i>participates</i>	user_id, workspace_id, role_id	Quan hệ 3 ngôi (M:N:1)
2	role_permissions	<i>contains</i>	role_id, permission_id	Quan hệ M:N
3	commit_file	<i>modifies</i>	commit_id, file_id	Quan hệ M:N
4	dependencies	<i>depends_on</i>	dependent_entity_id, target_entity_id	Đệ quy trở về code_entities
5	template_mappings	<i>maps_to</i>	placeholder_id, code_entity_id	Quan hệ M:N
6	grounding_evidence_code_entities	<i>depends_on</i>	evidence_id, code_entity_id	Quan hệ M:N
7	branch_commit	<i>includes</i>	branch_id, commit_id	Quan hệ N:N — một commit thuộc nhiều branch
8	pull_request_commit	<i>includes</i>	pr_id, commit_id	Quan hệ N:N — một commit thuộc nhiều PR

Bảng 3.3: Chuyển đổi Mối kết hợp M:N thành Bảng trung gian

STT	Bảng trung gian	Mối kết hợp gốc	Danh sách các Khóa ngoại (FK)	Ghi chú
9	commit_parent	has_parent	commit_id, parent_commit_id	Đệ quy trở về commits (merge commit nhiều cha)

4. Xử lý Thuộc tính đặc biệt (Phức hợp & Đa trị) (Bước 6)

Bảng 3.4: Xử lý Thuộc tính đặc biệt (Phức hợp & Đa trị)

STT	Loại thuộc tính	Thuộc tính gốc (ERD)	Giải pháp chuyển đổi	Cột / Bảng kết quả
1	Phức hợp	User.FullName	Phẳng hóa thành các cột	first_name, last_name trong users
2	Phức hợp	Commit.AuthorInfo	Phẳng hóa thành các cột	author_name, author_email trong commits
3	Phức hợp	CodeEntity.position	Phẳng hóa thành các cột	start_line, end_line trong code_entities
4	Đa trị	User.NotificationPreferences	Tách thành bảng phụ chứa FK	Bảng user_notification_preferences
5	Đa trị	GitHubConnection.scopes	Tách thành bảng phụ chứa FK	Bảng github_connection_scopes
6	Đa trị	PullRequest.labels	Tách thành bảng phụ chứa FK	Bảng pull_request_labels

2. Phần 2: Lược đồ Quan hệ Lô-gíc (Logical Schema)

2.1. Khai báo Chi tiết Bảng

A. Nhóm Quản trị & Kết nối

Bảng: users

Bảng chính (Thực thể mạnh: User)

Cột	Kiểu	Khóa	Ràng buộc
user_id	UUID	PK	NOT NULL
email		Unique	NOT NULL
password_hash			NULL
first_name			NULL
last_name			NULL
status			NOT NULL
created_at			NOT NULL
updated_at			NOT NULL
deleted_at			NULL

Bảng: workspaces

Bảng chính (Thực thể mạnh: Workspace)

Cột	Kiểu	Khóa	Ràng buộc
workspace_id	UUID	PK	NOT NULL
name			NOT NULL
description			NULL
status			NOT NULL
created_at			NOT NULL
updated_at			NOT NULL
deleted_at			NULL

Bảng: roles

Bảng chính (Thực thể mạnh: Role)

Cột	Kiểu	Khóa	Ràng buộc
role_id	UUID	PK	NOT NULL
role_name			NOT NULL
description			NULL
created_at			NOT NULL
updated_at			NOT NULL

Bảng: permissions

Bảng chính (Thực thể mạnh: Permission)

Cột	Kiểu	Khóa	Ràng buộc
permission_id	UUID	PK	NOT NULL
permission_code		Unique	NOT NULL
description			NULL
created_at			NOT NULL

Bảng: github_connections

Bảng phụ (Thực thể yếu – phụ thuộc User)

Cột	Kiểu	Khóa	Ràng buộc
connection_id	UUID	PK	NOT NULL
user_id		FK, Unique	NOT NULL
github_user_id			NOT NULL
github_username			NOT NULL
access_token			NOT NULL
created_at			NOT NULL
updated_at			NOT NULL

Bảng: user_notification_preferences

Bảng phụ đa trị

Cột	Kiểu	Khóa	Ràng buộc
preference_id	UUID	PK	NOT NULL
user_id		FK	NOT NULL
channel			NOT NULL

Bảng: github_connection_scopes

Bảng phụ đa trị

Cột	Kiểu	Khóa	Ràng buộc
scope_id	UUID	PK	NOT NULL
connection_id		FK	NOT NULL
scope_value			NOT NULL

Bảng: workspace_members

Bảng trung gian

Cột	Kiểu	Khóa	Ràng buộc
membership_id	UUID	PK	NOT NULL
user_id		FK	NOT NULL
workspace_id		FK	NOT NULL
role_id		FK	NOT NULL
joined_at			NOT NULL
status			NOT NULL

Bảng: role_permissions
Bảng trung gian

Cột	Kiểu	Khóa	Ràng buộc
role_permission_id	UUID	PK	NOT NULL
role_id		FK	NOT NULL
permission_id		FK	NOT NULL
granted_at			NOT NULL

Nhóm Mã nguồn & Cấu trúc Code

Bảng: repositories
Bảng chính (Thực thể mạnh: Repository)

Cột	Kiểu	Khóa	Ràng buộc
repository_id	UUID	PK	NOT NULL
workspace_id		FK	NOT NULL
github_connection_id		FK	NULL
name			NOT NULL
url			NOT NULL
default_branch			NULL
sync_status			NOT NULL
last_synced_at			NULL
created_at			NOT NULL
updated_at			NOT NULL

Bảng: branches
Bảng phụ (Thực thể yếu – phụ thuộc Repository)

Cột	Kiểu	Khóa	Ràng buộc
branch_id	UUID	PK	NOT NULL
repository_id		FK	NOT NULL
branch_name			NOT NULL
is_default			NOT NULL
created_at			NOT NULL
updated_at			NOT NULL

Bảng: commits

Bảng phụ (Thực thể yếu – phụ thuộc Repository)

Cột	Kiểu	Khóa	Ràng buộc
commit_id	UUID	PK	NOT NULL
repository_id		FK	NOT NULL
commit_hash			NOT NULL
author_name			NOT NULL
author_email			NOT NULL
message			NULL
committed_at			NOT NULL
created_at			NOT NULL

Bảng: pull_requests

Bảng phụ (Thực thể yếu – phụ thuộc Repository)

Cột	Kiểu	Khóa	Ràng buộc
pr_id	UUID	PK	NOT NULL
repository_id		FK	NOT NULL
pr_number			NOT NULL
title			NOT NULL
description			NULL
source_branch			NOT NULL
target_branch			NOT NULL
status			NOT NULL
created_at			NOT NULL
updated_at			NOT NULL
merged_at			NULL

Bảng: pull_request_labels

Bảng phụ đa trị

Cột	Kiểu	Khóa	Ràng buộc
label_id	UUID	PK	NOT NULL
pr_id		FK	NOT NULL
label_value			NOT NULL

Bảng: code_diffs

Bảng phụ (Thực thể yếu – phụ thuộc Commit)

Cột	Kiểu	Khóa	Ràng buộc
diff_id	UUID	PK	NOT NULL
commit_id		FK	NOT NULL
file_path			NOT NULL
change_type			NOT NULL
additions			NOT NULL
deletions			NOT NULL
diff_content			NULL
created_at			NOT NULL

Bảng: commit_file

Bảng trung gian

Cột	Kiểu	Khóa	Ràng buộc
change_id	UUID	PK	NOT NULL
commit_id		FK	NOT NULL
file_id		FK	NOT NULL
change_type			NOT NULL
lines_added			NOT NULL
lines_deleted			NOT NULL

Bảng: files

Bảng phụ (Thực thể yếu – phụ thuộc Repository)

Cột	Kiểu	Khóa	Ràng buộc
file_id	UUID	PK	NOT NULL
repository_id		FK	NOT NULL
file_path			NOT NULL
file_type			NULL
status			NOT NULL
created_at			NOT NULL
updated_at			NOT NULL

Bảng: code_entities

Bảng phụ (Thực thể yếu – phụ thuộc File)

Cột	Kiểu	Khóa	Ràng buộc
entity_id	UUID	PK	NOT NULL
file_id		FK	NOT NULL
entity_name			NOT NULL
entity_type			NOT NULL
signature			NULL
doc_comment			NULL
start_line			NOT NULL
end_line			NOT NULL
created_at			NOT NULL
updated_at			NOT NULL

Bảng: dependencies

Bảng trung gian đệ quy

Cột	Kiểu	Khóa	Ràng buộc
dependency_id	UUID	PK	NOT NULL
dependent_entity_id		FK	NOT NULL
target_entity_id		FK	NOT NULL
dependency_type			NOT NULL

Bảng: branch_commit

Bảng trung gian

Cột	Kiểu	Khóa	Ràng buộc
link_id	UUID	PK	NOT NULL
branch_id		FK	NOT NULL
commit_id		FK	NOT NULL

Bảng: pull_request_commit
Bảng trung gian

Cột	Kiểu	Khóa	Ràng buộc
link_id	UUID	PK	NOT NULL
pr_id		FK	NOT NULL
commit_id		FK	NOT NULL

Bảng: commit_parent
Bảng trung gian đệ quy

Cột	Kiểu	Khóa	Ràng buộc
link_id	UUID	PK	NOT NULL
commit_id		FK	NOT NULL
parent_commit_id		FK	NOT NULL

Nhóm Khuôn mẫu Tài liệu

Bảng: templates
Bảng chính (Thực thể mạnh: Template)

Cột	Kiểu	Khóa	Ràng buộc
template_id	UUID	PK	NOT NULL
workspace_id		FK	NOT NULL
name			NOT NULL
description			NULL
content_structure			NULL
creator_type			NOT NULL
status			NOT NULL
created_at			NOT NULL
updated_at			NOT NULL

Bảng: template_sections
Bảng phụ (Thực thể yếu – phụ thuộc Template)

Cột	Kiểu	Khóa	Ràng buộc
section_id	UUID	PK	NOT NULL
template_id		FK	NOT NULL
heading_title			NOT NULL
order_index			NOT NULL
prompt_instruction			NULL
created_at			NOT NULL
updated_at			NOT NULL

Bảng: placeholders

Bảng phụ (Thực thể yếu – phụ thuộc TemplateSection)

Cột	Kiểu	Khóa	Ràng buộc
placeholder_id	UUID	PK	NOT NULL
section_id		FK	NOT NULL
variable_name			NOT NULL
expected_data_type			NOT NULL
description			NULL
is_required			NOT NULL
created_at			NOT NULL
updated_at			NOT NULL

Bảng: template_mappings

Bảng trung gian

Cột	Kiểu	Khóa	Ràng buộc
mapping_id	UUID	PK	NOT NULL
placeholder_id		FK	NOT NULL
code_entity_id		FK	NOT NULL
extraction_rule			NULL
is_active			NOT NULL

Bảng: documents

Bảng chính (Thực thể mạnh: Document)

Cột	Kiểu	Khóa	Ràng buộc
document_id	UUID	PK	NOT NULL
workspace_id		FK	NOT NULL
author_user_id		FK	NOT NULL
title			NOT NULL
document_type			NOT NULL
status			NOT NULL
created_at			NOT NULL
updated_at			NOT NULL
deleted_at			NULL

Bảng: document_versions

Bảng phụ (Thực thể yếu – phụ thuộc Document)

Cột	Kiểu	Khóa	Ràng buộc
version_id	UUID	PK	NOT NULL
document_id		FK	NOT NULL
version_number			NOT NULL
content			NULL
change_summary			NULL
status			NOT NULL
ticket_codes			NULL
created_at			NOT NULL
updated_at			NOT NULL

Bảng: change_logs

Bảng phụ (Thực thể yếu – phụ thuộc Document Version)

Cột	Kiểu	Khóa	Ràng buộc
log_id	UUID	PK	NOT NULL
version_id		FK	NOT NULL
change_type			NOT NULL
change_description			NULL
diff_content			NULL
created_at			NOT NULL
updated_at			NOT NULL

3.0.2 Phân hệ minh chứng AI & Giám sát

Bảng: `grounding_evidences`

Bảng phụ (Thực thể yếu – phụ thuộc DocumentVersion)

Cột	Kiểu	Khóa	Ràng buộc
evidence_id	UUID	PK	NOT NULL
version_id		FK	NOT NULL
snippet_content			NULL
purpose			NOT NULL
created_at			NOT NULL

Bảng: `grounding_evidence_code_entities`

Bảng trung gian

Cột	Kiểu	Khóa	Ràng buộc
link_id	UUID	PK	NOT NULL
evidence_id		FK	NOT NULL
code_entity_id		FK	NOT NULL
similarity_score			NULL
match_type			NULL

Bảng: `drift_alerts`

Bảng phụ / sự kiện (phụ thuộc GroundingEvidence)

Cột	Kiểu	Khóa	Ràng buộc
alert_id	UUID	PK	NOT NULL
evidence_id		FK	NOT NULL
drift_type			NOT NULL
status			NOT NULL
detected_at			NOT NULL

Sau khi áp dụng thành công **Thuật toán 2.2** (Ánh xạ mô hình ERD Quan niệm sang Lược đồ Quan hệ), toàn bộ thực thể, thuộc tính và mối kết hợp trong hệ thống đã được chuyển đổi hoàn chỉnh sang **Lược đồ ERD ở mức Lô-gíc (Logical ERD)**.

Đầu ra đạt được sau bước này bao gồm:

- **Cấu trúc bảng chuẩn hóa:** Chuyển đổi toàn bộ thực thể và các mối kết hợp $M : N$ thành **31 bảng** dữ liệu độc lập với tên gọi tuân thủ chuẩn `snake_case`.

- **Xử lý thuộc tính phức hợp & đa trị:** Giải quyết triệt để các thuộc tính phức hợp, thuộc tính đa trị và chuyển đổi các mối kết hợp 3 ngôi (như cụm USER-WORKSPACE-ROLE) thành các thực thể liên kết hợp lệ.
- **Cơ chế Khóa chính & Khóa ngoại:** Đã đồng bộ hóa việc sử dụng Khóa chính đơn *Surrogate PK (UUID)* cho tất cả các bảng, đồng thời thiết lập mạng lưới Khóa ngoại (*FK*) chặt chẽ để duy trì ràng buộc toàn vẹn tham chiếu.
- **Đạt dạng chuẩn 3NF/BCNF:** Xác định tập phụ thuộc hàm (FDs) cho 4 phân hệ nghiệp vụ, chứng minh toàn bộ các bảng loại bỏ được rủi ro dư thừa dữ liệu và bất thường thao tác.

Chi tiết sơ đồ ERD mức Lô-gíc và mô tả cấu trúc các bảng được trình bày ngay dưới đây:

Phần 3: BÁO CÁO CHỨNG MINH CHUẨN HÓA 3NF

4.1. Bảng users

Phụ thuộc hàm (FD)	$f1: user_id \rightarrow email, password_hash, first_name, last_name, status, created_at, updated_at, deleted_at$ $f2: email \rightarrow user_id$
Chuẩn 1NF	Mọi cột nguyên tố, FullName/NotificationPreferences đã tách.
Chuẩn 2NF	PK đơn (<i>user_id</i>) không phụ thuộc một phần.
Chuẩn 3NF	Không thuộc tính không khóa nào xác định thuộc tính không khóa khác $\rightarrow$ đạt 3NF.

4.2. Bảng commits (Đã sửa theo Task 3)

Phụ thuộc hàm (FD)	$f1: commit_id \rightarrow repository_id, commit_hash, author_name, author_email, message, committed_at, created_at$
--------------------	---

Chuẩn 1NF	AuthorInfo đã phẳng hóa; mọi giá trị nguyên tố.
Chuẩn 2NF	PK đơn <code>commit_id</code> (UUID) → <code>repository_id</code> , <code>commit_hash...</code> đều phụ thuộc đầy đủ vào <code>commit_id</code> , không có phụ thuộc một phần.
Chuẩn 3NF	<code>author_name</code> và <code>author_email</code> không xác định lẫn nhau hay xác định <code>repository_id</code> hay <code>message</code> một cách bắc cầu → không có phụ thuộc bắc cầu. Bảng đạt 3NF.
Lưu ý	Sau khi bỏ <code>branch_id</code> / <code>author_user_id</code> / <code>pr_id</code> , bảng chỉ còn đúng 1 FK định danh (<code>repository_id</code>) → cấu trúc phụ thuộc hàm càng đơn giản, càng dễ chứng minh 3NF hơn bản trước.

4.3. Bảng `workspace_members` (3 ngôi, có thuộc tính riêng)

Phụ thuộc hàm (FD)	f1: <code>membership_id</code> → <code>user_id</code> , <code>workspace_id</code> , <code>role_id</code> , <code>joined_at</code> , <code>status</code>
Ràng buộc nghiệp vụ	<code>(user_id, workspace_id)</code> → <code>role_id</code> , ép buộc bằng <code>UNIQUE(user_id, workspace_id)</code> .
Chuẩn 1NF - 3NF	Surrogate PK (<code>membership_id</code>) khiến mọi thuộc tính phụ thuộc đầy đủ và không bắc cầu → đạt 3NF.

4.4. Bảng `dependencies` (đệ quy CodeEntity)

Phụ thuộc hàm (FD)	f1: <code>dependency_id</code> → <code>dependent_entity_id</code> , <code>target_entity_id</code> , <code>dependency_type</code>
Chuẩn 1NF - 3NF	PK đơn <code>dependency_id</code> , CHECK loại trừ tự tham chiếu, <code>dependency_type</code> chỉ phụ thuộc qua <code>dependency_id</code> , không bắc cầu → đạt 3NF.

4.5. Bảng `grounding_evidence_code_entities` (có thuộc tính đo lường)

Phụ thuộc hàm (FD)	$f1: link_id \rightarrow evidence_id, code_entity_id, similarity_score, match_type$
Chuẩn 1NF - 3NF	PK đơn <code>link_id</code> ; <code>similarity_score/match_type</code> không xác định lẫn nhau hay xác định <code>evidence_id/code_entity_id</code> → đạt 3NF.

4.6. Nhóm 3 bảng trung gian mới: `branch_commit`, `pull_request_commit`, `commit_parent`

Đặc điểm cấu trúc	Cả 3 bảng đều theo đúng khuôn mẫu đã chứng minh ở mục 4.3/4.4: Surrogate PK riêng (<code>link_id</code>), chỉ chứa 2 cột FK (không có thuộc tính mô tả riêng nào khác).
Ràng buộc	Với riêng <code>commit_parent</code> , ràng buộc <code>CHECK(commit_id $\neq$ parent_commit_id)</code> loại bỏ trường hợp tự tham chiếu vô nghĩa.
Chuẩn 1NF - 3NF	Không tồn tại thuộc tính không khóa để xét phụ thuộc bắc cầu, tự động đạt 2NF và 3NF.

4.7. Kết luận chung

Áp dụng cùng lập luận cho toàn bộ 35 bảng: khóa chính đơn, thuộc tính phức hợp/đa trị đã xử lý ở Bước 6, mọi mối kết hợp M:N/3-ngôi/đệ quy đều hiện thực bằng bảng trung gian Surrogate PK độc lập → **không tồn tại phụ thuộc một phần (loại trừ 2NF)** và **không tồn tại phụ thuộc bắc cầu (loại trừ 3NF)**.

Kết luận: Toàn bộ lược đồ quan hệ của phân hệ Living Docs (35 bảng, đã đồng bộ với Task 2 & Task 3 phiên bản mới nhất) đạt tiêu chuẩn 3NF, đảm bảo tính toàn vẹn dữ liệu và tối ưu hóa lưu trữ.

Phần III

Thiết Kế Vật Lý

Chương 4

Thiết Kế CSDL Vật Lý & DDL SQL

4.1 DDL

Trong giai đoạn thiết kế vật lý, lược đồ cơ sở dữ liệu được triển khai dưới dạng các câu lệnh DDL (Data Definition Language) trên hệ quản trị cơ sở dữ liệu Microsoft SQL Server. Các câu lệnh DDL dưới đây thực hiện việc tạo bảng, khóa chính, khóa ngoại, các ràng buộc toàn vẹn, chỉ mục và trigger nhằm đảm bảo cấu trúc dữ liệu phù hợp với mô hình thiết kế đã xây dựng. Việc triển khai này là cơ sở để hệ thống có thể lưu trữ, truy xuất và quản lý dữ liệu một cách nhất quán.

```
1 CREATE TABLE users (
2     user_id          UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
3     email             VARCHAR(255)     NOT NULL,
4     password_hash     VARCHAR(255)     NULL,
5     first_name        NVARCHAR(100)    NULL,
6     last_name         NVARCHAR(100)    NULL,
7     status            VARCHAR(20)      NOT NULL DEFAULT 'ACTIVE',
8     created_at        DATETIME2        NOT NULL DEFAULT GETDATE()
9 ,
10    updated_at        DATETIME2        NOT NULL DEFAULT GETDATE()
11 ,
12    deleted_at        DATETIME2        NULL,
13
14    CONSTRAINT pk_users PRIMARY KEY (user_id),
15    CONSTRAINT uq_users_email UNIQUE (email),
16    CONSTRAINT chk_users_status CHECK (status IN ('ACTIVE', 'INACTIVE', 'SUSPENDED'))
17 );
```

```

16
17 CREATE TABLE roles (
18     role_id          UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
19     role_name        VARCHAR(50)      NOT NULL,
20     description      NVARCHAR(MAX)    NULL,
21     created_at       DATETIME2        NOT NULL DEFAULT GETDATE()
22     ,
23     updated_at       DATETIME2        NOT NULL DEFAULT GETDATE()
24     ,
25     CONSTRAINT pk_roles PRIMARY KEY (role_id),
26     CONSTRAINT uq_roles_name UNIQUE (role_name)
27 );
28
29 CREATE TABLE permissions (
30     permission_id    UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
31     permission_code  VARCHAR(50)      NOT NULL,
32     description      NVARCHAR(MAX)    NULL,
33     created_at       DATETIME2        NOT NULL DEFAULT GETDATE()
34     ,
35     CONSTRAINT pk_permissions PRIMARY KEY (permission_id),
36     CONSTRAINT uq_permissions_code UNIQUE (permission_code)
37 );
38
39 CREATE TABLE role_permissions (
40     role_permission_id UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID()
41     ,
42     role_id            UNIQUEIDENTIFIER NOT NULL,
43     permission_id      UNIQUEIDENTIFIER NOT NULL,
44     granted_at         DATETIME2        NOT NULL DEFAULT
45     GETDATE(),
46     CONSTRAINT pk_role_permissions PRIMARY KEY (
47     role_permission_id),
48     CONSTRAINT uq_role_permissions_pair UNIQUE (role_id,
49     permission_id),
50     CONSTRAINT fk_role_permissions_role
51     FOREIGN KEY (role_id) REFERENCES roles (role_id) ON
52     DELETE CASCADE,
53     CONSTRAINT fk_role_permissions_permission
54     FOREIGN KEY (permission_id) REFERENCES permissions (
55     permission_id) ON DELETE CASCADE
56 );
57
58 CREATE INDEX idx_role_permissions_role_id
59 ON role_permissions(role_id);
60
61 CREATE TABLE user_notification_preferences (

```

```

56     preference_id    UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
57     user_id          UNIQUEIDENTIFIER NOT NULL,
58     channel          VARCHAR(20)      NOT NULL,
59     created_at       DATETIME2        NOT NULL DEFAULT GETDATE()
60 ,
61     CONSTRAINT pk_user_notification_preferences PRIMARY KEY (
62     preference_id),
63     CONSTRAINT fk_user_notification_pref_user
64     FOREIGN KEY (user_id) REFERENCES users (user_id) ON
65     DELETE CASCADE
66 );
67
68 CREATE TABLE workspaces (
69     workspace_id    UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
70     name            NVARCHAR(150)    NOT NULL,
71     description     NVARCHAR(MAX)    NULL,
72     status          VARCHAR(20)      NOT NULL DEFAULT 'ACTIVE',
73     created_at      DATETIME2        NOT NULL DEFAULT GETDATE()
74 ,
75     updated_at      DATETIME2        NOT NULL DEFAULT GETDATE()
76 ,
77     deleted_at      DATETIME2        NULL,
78
79     CONSTRAINT pk_workspaces PRIMARY KEY (workspace_id),
80     CONSTRAINT chk_workspace_status CHECK (status IN ('ACTIVE',
81     'ARCHIVED'))
82 );
83
84 CREATE TABLE workspace_members (
85     membership_id   UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
86     user_id         UNIQUEIDENTIFIER NOT NULL,
87     workspace_id    UNIQUEIDENTIFIER NOT NULL,
88     role_id         UNIQUEIDENTIFIER NOT NULL,
89     joined_at       DATETIME2        NOT NULL DEFAULT GETDATE()
90 ,
91     status          VARCHAR(20)      NOT NULL DEFAULT 'ACTIVE',
92
93     CONSTRAINT pk_workspace_members PRIMARY KEY (membership_id)
94 ,
95     CONSTRAINT uq_workspace_members_user_workspace UNIQUE (
96     user_id, workspace_id),
97     CONSTRAINT fk_workspace_members_user
98     FOREIGN KEY (user_id) REFERENCES users (user_id) ON
99     DELETE CASCADE,
100    CONSTRAINT fk_workspace_members_workspace
101    FOREIGN KEY (workspace_id) REFERENCES workspaces (
102    workspace_id) ON DELETE CASCADE,
103    CONSTRAINT fk_workspace_members_role

```

```

94         FOREIGN KEY (role_id) REFERENCES roles (role_id)
95     );
96
97     CREATE INDEX idx_workspace_members_workspace_id
98     ON workspace_members(workspace_id);
99
100    CREATE INDEX idx_workspace_members_user_id
101    ON workspace_members(user_id);
102
103    CREATE TABLE github_connections (
104        connection_id    UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
105        workspace_id     UNIQUEIDENTIFIER NOT NULL,
106        user_id          UNIQUEIDENTIFIER NULL,
107        provider         VARCHAR(50)      NOT NULL DEFAULT 'GITHUB',
108        github_user_id   VARCHAR(50)      NULL,
109        github_username  NVARCHAR(100)    NOT NULL,
110        access_token     VARCHAR(500)     NOT NULL,
111        sync_status      VARCHAR(20)      NOT NULL DEFAULT 'IDLE',
112        created_at       DATETIME2        NOT NULL DEFAULT GETDATE()
113    ,
114        updated_at       DATETIME2        NULL,
115
116        CONSTRAINT pk_github_connections PRIMARY KEY (connection_id)
117    ),
118    CONSTRAINT fk_github_connections_workspace
119        FOREIGN KEY (workspace_id) REFERENCES workspaces(
120        workspace_id) ON DELETE CASCADE,
121    CONSTRAINT fk_github_connections_user
122        FOREIGN KEY (user_id) REFERENCES users(user_id) ON
123        DELETE SET NULL,
124    CONSTRAINT chk_sync_status CHECK (
125        sync_status IN ('IDLE', 'SYNCING', 'SUCCESS', 'FAILED')
126    )
127 );
128
129    CREATE INDEX idx_github_connections_workspace_id
130    ON github_connections(workspace_id);
131
132    CREATE INDEX idx_github_connections_user_id
133    ON github_connections(user_id);
134
135    CREATE TABLE github_connection_scopes (
136        scope_id         UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
137        connection_id     UNIQUEIDENTIFIER NOT NULL,
138        scope_value       VARCHAR(100)    NOT NULL,
139        created_at       DATETIME2        NOT NULL DEFAULT GETDATE()
140    ,

```

```

137     CONSTRAINT pk_github_connection_scopes PRIMARY KEY (
138         scope_id),
139     CONSTRAINT uq_connection_scope UNIQUE (connection_id,
140         scope_value),
141     CONSTRAINT fk_github_scopes_connection
142         FOREIGN KEY (connection_id) REFERENCES
143         github_connections(connection_id) ON DELETE CASCADE
144 );
145
146 CREATE TABLE repositories (
147     repository_id UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
148     workspace_id UNIQUEIDENTIFIER NOT NULL,
149     name VARCHAR(150) NOT NULL,
150     url VARCHAR(255) NOT NULL,
151     default_branch VARCHAR(50) NOT NULL DEFAULT 'main',
152     last_synced_at DATETIME2 NULL,
153     created_at DATETIME2 NOT NULL DEFAULT GETDATE()
154 ,
155     updated_at DATETIME2 NULL,
156
157     CONSTRAINT pk_repositories PRIMARY KEY (repository_id),
158     CONSTRAINT uq_repository_workspace_name UNIQUE (
159         workspace_id, name),
160     CONSTRAINT fk_repositories_workspace
161         FOREIGN KEY (workspace_id) REFERENCES workspaces(
162         workspace_id) ON DELETE CASCADE
163 );
164
165 CREATE INDEX idx_repositories_workspace_id
166 ON repositories(workspace_id);
167
168 CREATE TABLE branches (
169     branch_id UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
170     repository_id UNIQUEIDENTIFIER NOT NULL,
171     branch_name VARCHAR(100) NOT NULL,
172     is_default BIT NOT NULL DEFAULT 0,
173     created_at DATETIME2 NOT NULL DEFAULT GETDATE()
174 ,
175     updated_at DATETIME2 NULL,
176
177     CONSTRAINT pk_branches PRIMARY KEY (branch_id),
178     CONSTRAINT uq_branch_repository_name UNIQUE (repository_id,
179         branch_name),
180     CONSTRAINT fk_branches_repository
181         FOREIGN KEY (repository_id) REFERENCES repositories(
182         repository_id) ON DELETE CASCADE
183 );
184
185 CREATE INDEX idx_branches_repository_id

```

```

177 ON branches(repository_id);
178
179 CREATE TABLE pull_requests (
180     pr_id                UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
181     repository_id        UNIQUEIDENTIFIER NOT NULL,
182     pr_number            INT               NOT NULL,
183     title                NVARCHAR(500)    NOT NULL,
184     description           NVARCHAR(MAX)    NULL,
185     source_branch        VARCHAR(255)     NULL,
186     target_branch        VARCHAR(255)     NULL,
187     status               VARCHAR(20)      NOT NULL DEFAULT 'OPEN',
188     created_at           DATETIME2        NOT NULL DEFAULT GETDATE()
189 ,
190     updated_at           DATETIME2        NULL,
191     merged_at            DATETIME2        NULL,
192
193     CONSTRAINT pk_pull_requests PRIMARY KEY (pr_id),
194     CONSTRAINT uq_pull_requests_repository_pr UNIQUE (
195         repository_id, pr_number),
196     CONSTRAINT fk_pull_requests_repository
197         FOREIGN KEY (repository_id) REFERENCES repositories(
198             repository_id),
199     CONSTRAINT chk_pull_requests_status
200         CHECK (status IN ('OPEN', 'CLOSED', 'MERGED'))
201 );
202
203 CREATE INDEX idx_pull_requests_repository_id
204 ON pull_requests(repository_id);
205
206 CREATE INDEX idx_pull_requests_status
207 ON pull_requests(status);
208
209 CREATE INDEX idx_pull_requests_created_at
210 ON pull_requests(created_at);
211
212 CREATE TABLE pull_request_labels (
213     pull_request_label_id UNIQUEIDENTIFIER NOT NULL DEFAULT
214     NEWID(),
215     pr_id                UNIQUEIDENTIFIER NOT NULL,
216     label_name           NVARCHAR(100)    NOT NULL,
217     label_color          VARCHAR(20)      NULL,
218
219     CONSTRAINT pk_pull_request_labels PRIMARY KEY (
220         pull_request_label_id),
221     CONSTRAINT uq_pull_request_labels UNIQUE (pr_id, label_name
222     ),
223     CONSTRAINT fk_pull_request_labels_pr
224         FOREIGN KEY (pr_id) REFERENCES pull_requests(pr_id) ON
225         DELETE CASCADE

```

```

219 );
220
221 CREATE INDEX idx_pull_request_labels_pr_id
222 ON pull_request_labels(pr_id);
223
224 CREATE TABLE commits (
225     commit_id        UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
226     repository_id    UNIQUEIDENTIFIER NOT NULL,
227     branch_id        UNIQUEIDENTIFIER NOT NULL,
228     pr_id            UNIQUEIDENTIFIER NULL,
229     commit_hash       VARCHAR(255)     NOT NULL,
230     author_name       NVARCHAR(255)    NULL,
231     author_email      VARCHAR(255)     NULL,
232     message           NVARCHAR(MAX)    NULL,
233     committed_at      DATETIME2        NOT NULL,
234     created_at        DATETIME2        NOT NULL DEFAULT GETDATE()
235 ,
236     updated_at        DATETIME2        NULL,
237
238     CONSTRAINT pk_commits PRIMARY KEY (commit_id),
239     CONSTRAINT uq_commits_repository_hash UNIQUE (repository_id
240 , commit_hash),
241     CONSTRAINT fk_commits_repository
242     FOREIGN KEY (repository_id) REFERENCES repositories(
243 repository_id),
244     CONSTRAINT fk_commits_branch
245     FOREIGN KEY (branch_id) REFERENCES branches(branch_id),
246     CONSTRAINT fk_commits_pull_request
247     FOREIGN KEY (pr_id) REFERENCES pull_requests(pr_id) ON
248 DELETE SET NULL
249 );
250
251 CREATE INDEX idx_commits_repository_id
252 ON commits(repository_id);
253
254 CREATE INDEX idx_commits_branch_id
255 ON commits(branch_id);
256
257 CREATE INDEX idx_commits_pr_id
258 ON commits(pr_id);
259
260 CREATE INDEX idx_commits_committed_at
261 ON commits(committed_at);
262
263 CREATE TABLE files (
264     file_id          UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
265     repository_id    UNIQUEIDENTIFIER NOT NULL,
266     file_path        VARCHAR(1000)    NOT NULL,
267     file_type        VARCHAR(50)      NULL,

```

```

264     status          VARCHAR(20)          NOT NULL DEFAULT 'ACTIVE',
265     created_at      DATETIME2            NOT NULL DEFAULT GETDATE()
266 ,
267     updated_at      DATETIME2            NULL,
268
269     CONSTRAINT pk_files PRIMARY KEY (file_id),
270     CONSTRAINT uq_files_repository_file_path UNIQUE (
271 repository_id, file_path),
272     CONSTRAINT fk_files_repository
273     FOREIGN KEY (repository_id) REFERENCES repositories(
274 repository_id) ON DELETE CASCADE,
275     CONSTRAINT chk_files_status
276     CHECK (status IN ('ACTIVE', 'DELETED', 'RENAMED'))
277 );
278
279 CREATE INDEX idx_files_repository_id
280 ON files(repository_id);
281
282 CREATE INDEX idx_files_file_path
283 ON files(file_path);
284
285 CREATE TABLE code_diffs (
286     diff_id          UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
287     commit_id        UNIQUEIDENTIFIER NOT NULL,
288     pr_id            UNIQUEIDENTIFIER NULL,
289     file_id          UNIQUEIDENTIFIER NOT NULL,
290     file_path        VARCHAR(1000)     NULL,
291     change_type      VARCHAR(20)       NOT NULL,
292     additions        INT               NULL,
293     deletions        INT               NULL,
294     diff_content     NVARCHAR(MAX)     NULL,
295     created_at      DATETIME2            NOT NULL DEFAULT GETDATE()
296 ,
297
298     CONSTRAINT pk_code_diffs PRIMARY KEY (diff_id),
299     CONSTRAINT uq_code_diffs_commit_file UNIQUE (commit_id,
300 file_id),
301     CONSTRAINT fk_code_diffs_commit
302     FOREIGN KEY (commit_id) REFERENCES commits(commit_id)
303 ON DELETE CASCADE,
304     CONSTRAINT fk_code_diffs_pull_request
305     FOREIGN KEY (pr_id) REFERENCES pull_requests(pr_id) ON
306 DELETE SET NULL,
307     CONSTRAINT fk_code_diffs_file
308     FOREIGN KEY (file_id) REFERENCES files(file_id),
309     CONSTRAINT chk_code_diffs_change_type CHECK (
310 change_type IN ('ADDED', 'MODIFIED', 'DELETED', '
311 RENAMED'))
312 )

```

```

305 );
306
307 CREATE INDEX idx_code_diffs_commit_id
308 ON code_diffs(commit_id);
309
310 CREATE INDEX idx_code_diffs_file_id
311 ON code_diffs(file_id);
312
313 CREATE TABLE code_entities (
314     code_entity_id    UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
315     file_id           UNIQUEIDENTIFIER NOT NULL,
316     entity_name        NVARCHAR(255)    NOT NULL,
317     entity_type        VARCHAR(50)      NOT NULL,
318     signature          NVARCHAR(MAX)    NULL,
319     doc_comment        NVARCHAR(MAX)    NULL,
320     start_line         INT              NULL,
321     end_line           INT              NULL,
322     created_at         DATETIME2        NOT NULL DEFAULT GETDATE()
323 ,
324     updated_at         DATETIME2        NULL,
325
326     CONSTRAINT pk_code_entities PRIMARY KEY (code_entity_id),
327     CONSTRAINT uq_code_entities
328         UNIQUE (file_id, entity_name, entity_type, start_line),
329     CONSTRAINT fk_code_entities_file
330         FOREIGN KEY (file_id) REFERENCES files(file_id) ON
331         DELETE CASCADE,
332     CONSTRAINT chk_code_entities_entity_type CHECK (
333         entity_type IN (
334             'CLASS', 'INTERFACE', 'FUNCTION', 'METHOD', '
335             VARIABLE',
336             'CONSTANT', 'MODULE', 'PACKAGE', 'ENUM', 'STRUCT'
337         )
338     ),
339     CONSTRAINT chk_code_entities_line_range CHECK (
340         start_line IS NULL OR end_line IS NULL OR start_line <=
341         end_line
342     )
343 );
344
345 CREATE INDEX idx_code_entities_file_id
346 ON code_entities(file_id);
347
348 CREATE INDEX idx_code_entities_entity_name
349 ON code_entities(entity_name);
350
351 CREATE TABLE dependencies (
352     dependency_id      UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID
353     (),

```

```

349     source_entity_id    UNIQUEIDENTIFIER NOT NULL,
350     target_entity_id    UNIQUEIDENTIFIER NOT NULL,
351     dependency_type      VARCHAR(50)      NOT NULL,
352     created_at           DATETIME2        NOT NULL DEFAULT
353     GETDATE(),
354
355     CONSTRAINT pk_dependencies PRIMARY KEY (dependency_id),
356     CONSTRAINT uq_dependencies
357         UNIQUE (source_entity_id, target_entity_id,
358         dependency_type),
359     CONSTRAINT fk_dependencies_source_entity
360         FOREIGN KEY (source_entity_id) REFERENCES code_entities
361         (code_entity_id),
362     CONSTRAINT fk_dependencies_target_entity
363         FOREIGN KEY (target_entity_id) REFERENCES code_entities
364         (code_entity_id),
365     CONSTRAINT chk_dependencies_dependency_type CHECK (
366         dependency_type IN (
367             'CALLS', 'IMPORTS', 'INHERITS', 'IMPLEMENTS',
368             'USES', 'REFERENCES', 'DEPENDS_ON'
369         )
370     ),
371     CONSTRAINT chk_dependencies_no_self_reference
372         CHECK (source_entity_id <> target_entity_id)
373 );
374
375 CREATE INDEX idx_dependencies_source_entity_id
376 ON dependencies(source_entity_id);
377
378 CREATE INDEX idx_dependencies_target_entity_id
379 ON dependencies(target_entity_id);
380
381 CREATE TABLE templates (
382     template_id    UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID
383     ( ),
384     workspace_id   UNIQUEIDENTIFIER NOT NULL,
385     name           NVARCHAR(150)     NOT NULL,
386     description    NVARCHAR(MAX)     NULL,
387     content_structure NVARCHAR(MAX)  NULL,
388     creator_type   VARCHAR(20)       NOT NULL,
389     status         VARCHAR(20)       NOT NULL DEFAULT '
390     DRAFT',
391     created_at     DATETIME2          NOT NULL DEFAULT
392     GETDATE(),
393     updated_at     DATETIME2          NOT NULL DEFAULT
394     GETDATE(),
395
396     CONSTRAINT pk_templates PRIMARY KEY (template_id),

```

```

389     CONSTRAINT uq_templates_workspace_name UNIQUE (workspace_id
, name),
390     CONSTRAINT fk_templates_workspace
391         FOREIGN KEY (workspace_id) REFERENCES workspaces (
workspace_id) ON DELETE CASCADE,
392     CONSTRAINT ck_templates_creator_type
393         CHECK (creator_type IN ('SYSTEM', 'USER', 'AI')),
394     CONSTRAINT ck_templates_status
395         CHECK (status IN ('DRAFT', 'ACTIVE', 'ARCHIVED')),
396     CONSTRAINT ck_templates_name_not_blank
397         CHECK (LEN(LTRIM(RTRIM(name))) > 0),
398     CONSTRAINT ck_templates_updated_after_created
399         CHECK (updated_at >= created_at)
400 );
401
402 CREATE INDEX idx_templates_workspace_id
403 ON templates (workspace_id);
404
405 CREATE INDEX idx_templates_status
406 ON templates (status);
407
408 CREATE TABLE template_sections (
409     section_id          UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID
(),
410     template_id         UNIQUEIDENTIFIER NOT NULL,
411     heading_title       NVARCHAR(255)    NOT NULL,
412     order_index        INT               NOT NULL,
413     prompt_instruction  NVARCHAR(MAX)    NULL,
414     created_at         DATETIME2        NOT NULL DEFAULT
GETDATE(),
415     updated_at         DATETIME2        NOT NULL DEFAULT
GETDATE(),
416
417     CONSTRAINT pk_template_sections PRIMARY KEY (section_id),
418     CONSTRAINT uq_template_sections_order
419         UNIQUE (template_id, order_index),
420     CONSTRAINT fk_template_sections_template
421         FOREIGN KEY (template_id) REFERENCES templates (
template_id) ON DELETE CASCADE,
422     CONSTRAINT ck_template_sections_order_index
423         CHECK (order_index >= 0),
424     CONSTRAINT ck_template_sections_heading_not_blank
425         CHECK (LEN(LTRIM(RTRIM(heading_title))) > 0)
426 );
427
428 CREATE TABLE placeholders (
429     placeholder_id      UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID
(),
430     section_id         UNIQUEIDENTIFIER NOT NULL,

```

```

431     variable_name      VARCHAR(100)      NOT NULL,
432     expected_data_type VARCHAR(30)       NOT NULL,
433     description        NVARCHAR(MAX)     NULL,
434     is_required        BIT               NOT NULL DEFAULT 1,
435     created_at         DATETIME2         NOT NULL DEFAULT
GETDATE(),
436     updated_at         DATETIME2         NOT NULL DEFAULT
GETDATE(),
437
438     CONSTRAINT pk_placeholders PRIMARY KEY (placeholder_id),
439     CONSTRAINT uq_placeholders_section_variable
440         UNIQUE (section_id, variable_name),
441     CONSTRAINT fk_placeholders_section
442         FOREIGN KEY (section_id) REFERENCES template_sections (
section_id) ON DELETE CASCADE,
443     CONSTRAINT ck_placeholders_expected_data_type CHECK (
444         expected_data_type IN (
445             'STRING', 'NUMBER', 'BOOLEAN', 'DATE',
446             'LIST', 'OBJECT', 'CODE'
447         )
448     ),
449     CONSTRAINT ck_placeholders_variable_name_format CHECK (
450         variable_name NOT LIKE '%[^a-zA-Z0-9_]%'
451         AND variable_name NOT LIKE '[0-9]%'
452     )
453 );
454
455 CREATE TABLE template_mappings (
456     mapping_id          UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID
(),
457     placeholder_id      UNIQUEIDENTIFIER NOT NULL,
458     code_entity_id      UNIQUEIDENTIFIER NOT NULL,
459     extraction_rule     NVARCHAR(MAX)    NULL,
460     is_active          BIT               NOT NULL DEFAULT 1,
461
462     CONSTRAINT pk_template_mappings PRIMARY KEY (mapping_id),
463     CONSTRAINT uq_template_mappings_pair
464         UNIQUE (placeholder_id, code_entity_id),
465     CONSTRAINT fk_template_mappings_placeholder
466         FOREIGN KEY (placeholder_id) REFERENCES placeholders (
placeholder_id) ON DELETE CASCADE,
467     CONSTRAINT fk_template_mappings_code_entity
468         FOREIGN KEY (code_entity_id) REFERENCES code_entities (
code_entity_id) ON DELETE CASCADE
469 );
470
471 CREATE INDEX idx_template_mappings_code_entity
472 ON template_mappings (code_entity_id);
473

```

```

474 CREATE INDEX idx_template_mappings_active
475 ON template_mappings (placeholder_id)
476 WHERE is_active = 1;
477
478 CREATE TABLE documents (
479     document_id          UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID
480     ( ),
481     workspace_id         UNIQUEIDENTIFIER NOT NULL ,
482     author_user_id       UNIQUEIDENTIFIER NOT NULL ,
483     title                 NVARCHAR(255)     NOT NULL ,
484     document_type        VARCHAR(30)        NOT NULL ,
485     status                VARCHAR(20)        NOT NULL DEFAULT '
DRAFT' ,
486     created_at           DATETIME2          NOT NULL DEFAULT
GETDATE() ,
487     updated_at           DATETIME2          NOT NULL DEFAULT
GETDATE() ,
488     deleted_at           DATETIME2          NULL ,
489
490     CONSTRAINT pk_documents PRIMARY KEY (document_id),
491     CONSTRAINT fk_documents_workspace
492     FOREIGN KEY (workspace_id) REFERENCES workspaces (
workspace_id) ON DELETE CASCADE ,
493     CONSTRAINT fk_documents_author
494     FOREIGN KEY (author_user_id) REFERENCES users (user_id)
495
496     ,
497     CONSTRAINT ck_documents_type CHECK (
498     document_type IN (
499         'SRS', 'API_DOC', 'USER_GUIDE',
500         'RELEASE_NOTE', 'TECH_DESIGN', 'OTHER'
501     )
502     ),
503     CONSTRAINT ck_documents_status CHECK (
504     status IN ('DRAFT', 'IN_REVIEW', 'PUBLISHED', 'ARCHIVED'
505     ))
506     ),
507     CONSTRAINT ck_documents_title_not_blank
508     CHECK (LEN(LTRIM(RTRIM(title))) > 0),
509     CONSTRAINT ck_documents_deleted_after_created
510     CHECK (deleted_at IS NULL OR deleted_at >= created_at)
511 );
512
513 CREATE UNIQUE INDEX uq_documents_workspace_title_active
514 ON documents (workspace_id, title)
515 WHERE deleted_at IS NULL;
516
517 CREATE INDEX idx_documents_workspace_id
518 ON documents (workspace_id);
519

```

```

516 CREATE INDEX idx_documents_author
517 ON documents (author_user_id);
518
519 CREATE INDEX idx_documents_status
520 ON documents (status);
521
522 CREATE TABLE document_versions (
523     version_id          UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID
524     (),
525     document_id         UNIQUEIDENTIFIER NOT NULL,
526     version_number      VARCHAR(20)      NOT NULL,
527     content             NVARCHAR(MAX)    NULL,
528     change_summary      NVARCHAR(MAX)    NULL,
529     status              VARCHAR(20)      NOT NULL DEFAULT '
530     DRAFT',
531     ticket_codes        VARCHAR(500)     NULL,
532     created_at          DATETIME2        NOT NULL DEFAULT
533     GETDATE(),
534     updated_at          DATETIME2        NOT NULL DEFAULT
535     GETDATE(),
536
537     CONSTRAINT pk_document_versions PRIMARY KEY (version_id),
538     CONSTRAINT uq_document_versions_number
539     UNIQUE (document_id, version_number),
540     CONSTRAINT fk_document_versions_document
541     FOREIGN KEY (document_id) REFERENCES documents (
542     document_id) ON DELETE CASCADE,
543     CONSTRAINT ck_document_versions_status CHECK (
544     status IN (
545         'DRAFT', 'IN_REVIEW', 'APPROVED',
546         'REJECTED', 'PUBLISHED'
547     )
548 )
549 );
550
551 CREATE INDEX idx_document_versions_status
552 ON document_versions (status);
553
554 CREATE INDEX idx_document_versions_created_at
555 ON document_versions (document_id, created_at DESC);
556
557 CREATE TABLE change_logs (
558     log_id              UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID
559     (),
560     version_id          UNIQUEIDENTIFIER NOT NULL,
561     change_description   NVARCHAR(MAX)    NULL,
562     diff_content        NVARCHAR(MAX)    NULL,
563     created_at          DATETIME2        NOT NULL DEFAULT
564     GETDATE(),

```

```

558     updated_at          DATETIME2          NOT NULL DEFAULT
        GETDATE(),
559
560     CONSTRAINT pk_change_logs PRIMARY KEY (log_id),
561     CONSTRAINT fk_change_logs_version
562         FOREIGN KEY (version_id) REFERENCES document_versions (
        version_id) ON DELETE CASCADE
563 );
564
565 CREATE INDEX idx_change_logs_version_id
566 ON change_logs (version_id);
567
568 CREATE TABLE reviews (
569     review_id            UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
570     version_id           UNIQUEIDENTIFIER NOT NULL,
571     user_id              UNIQUEIDENTIFIER NOT NULL,
572     status               VARCHAR(20)       NOT NULL DEFAULT 'PENDING'
573 ,
574     decision             VARCHAR(20)       NULL,
575     summary_notes        NVARCHAR(MAX)     NULL,
576     created_at           DATETIME2         NOT NULL DEFAULT GETDATE()
577 ,
578     updated_at           DATETIME2         NULL,
579
580     CONSTRAINT pk_reviews PRIMARY KEY (review_id),
581     CONSTRAINT fk_reviews_version
582         FOREIGN KEY (version_id) REFERENCES document_versions (
        version_id) ON DELETE CASCADE,
583     CONSTRAINT fk_reviews_user
584         FOREIGN KEY (user_id) REFERENCES users(user_id),
585     CONSTRAINT chk_reviews_status
586         CHECK (status IN ('PENDING', 'IN_PROGRESS', 'COMPLETED'
        )),
587     CONSTRAINT chk_reviews_decision
588         CHECK (
589             decision IS NULL
590             OR decision IN ('APPROVED', 'REJECTED', '
        CHANGES_REQUESTED')
591         )
592 );
593
594 CREATE INDEX idx_reviews_version_id
595 ON reviews(version_id);
596
597 CREATE INDEX idx_reviews_user_id
598 ON reviews(user_id);
599
600 CREATE TABLE review_comments (
        comment_id          UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),

```

```

600     review_id        UNIQUEIDENTIFIER NOT NULL,
601     user_id          UNIQUEIDENTIFIER NOT NULL,
602     content          NVARCHAR(MAX)    NOT NULL,
603     is_resolved      BIT              NOT NULL DEFAULT 0,
604     created_at       DATETIME2        NOT NULL DEFAULT GETDATE()
605 ,
606     updated_at       DATETIME2        NULL,
607
608     CONSTRAINT pk_review_comments PRIMARY KEY (comment_id),
609     CONSTRAINT fk_comments_review FOREIGN KEY (review_id) REFERENCES reviews(review_id)
610 ON DELETE CASCADE,
611     CONSTRAINT fk_comments_user FOREIGN KEY (user_id) REFERENCES users(user_id)
612 );
613
614 CREATE INDEX idx_review_comments_review_id
615 ON review_comments(review_id);
616
617 CREATE TABLE approvals (
618     approval_id       UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
619     version_id        UNIQUEIDENTIFIER NOT NULL,
620     user_id           UNIQUEIDENTIFIER NOT NULL,
621     decision          VARCHAR(20)      NOT NULL,
622     rejection_reason  NVARCHAR(MAX)    NULL,
623     approved_at       DATETIME2        NOT NULL DEFAULT GETDATE()
624 ,
625
626     CONSTRAINT pk_approvals PRIMARY KEY (approval_id),
627     CONSTRAINT fk_approvals_version FOREIGN KEY (version_id) REFERENCES document_versions(
628 version_id) ON DELETE CASCADE,
629     CONSTRAINT fk_approvals_user FOREIGN KEY (user_id) REFERENCES users(user_id),
630     CONSTRAINT chk_approvals_decision CHECK (decision IN ('APPROVED', 'REJECTED'))
631 );
632
633
634 CREATE INDEX idx_approvals_version_id
635 ON approvals(version_id);
636
637 CREATE TABLE grounding_evidences (
638     evidence_id       UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
639     version_id        UNIQUEIDENTIFIER NOT NULL,
640     code_entity_id    UNIQUEIDENTIFIER NULL,
641     snippet_content   NVARCHAR(MAX)    NOT NULL,
642     purpose           VARCHAR(30)      NOT NULL DEFAULT '
643 HALLUCINATION_PREVENTION',
644     match_type        VARCHAR(50)      NULL,

```

```

644     similarity_score FLOAT          NULL,
645     created_at      DATETIME2      NOT NULL DEFAULT GETDATE
646     (),
647     CONSTRAINT pk_grounding_evidences PRIMARY KEY (evidence_id)
648 ,
649     CONSTRAINT fk_evidences_version
650     FOREIGN KEY (version_id) REFERENCES document_versions (
651     version_id) ON DELETE CASCADE,
652     CONSTRAINT fk_evidences_code_entity
653     FOREIGN KEY (code_entity_id) REFERENCES code_entities(
654     code_entity_id) ON DELETE SET NULL,
655     CONSTRAINT chk_evidences_purpose CHECK (
656     purpose IN (
657     'HALLUCINATION_PREVENTION',
658     'CODE_REFERENCE',
659     'AST_VERIFICATION'
660     )
661 )
662 );
663
664 CREATE INDEX idx_grounding_evidences_version_id
665 ON grounding_evidences(version_id);
666
667 CREATE INDEX idx_grounding_evidences_code_entity_id
668 ON grounding_evidences(code_entity_id);
669
670 CREATE TABLE drift_alerts (
671     alert_id          UNIQUEIDENTIFIER NOT NULL DEFAULT NEWID(),
672     evidence_id       UNIQUEIDENTIFIER NOT NULL,
673     repository_id     UNIQUEIDENTIFIER NOT NULL,
674     drift_type        VARCHAR(20)      NOT NULL,
675     status            VARCHAR(20)      NOT NULL DEFAULT 'OPEN',
676     detected_at       DATETIME2        NOT NULL DEFAULT GETDATE()
677 ,
678     CONSTRAINT pk_drift_alerts PRIMARY KEY (alert_id),
679     CONSTRAINT fk_alerts_evidence
680     FOREIGN KEY (evidence_id) REFERENCES
681     grounding_evidences(evidence_id) ON DELETE CASCADE,
682     CONSTRAINT fk_alerts_repository
683     FOREIGN KEY (repository_id) REFERENCES repositories(
684     repository_id),
685     CONSTRAINT chk_alerts_type
686     CHECK (drift_type IN ('REFERENTIAL', 'SIGNATURE', '
687     SEMANTIC', 'OUTDATED')),
688     CONSTRAINT chk_alerts_status
689     CHECK (status IN ('OPEN', 'RESOLVED', 'IGNORED'))
690 );

```

```

685
686 CREATE INDEX idx_drift_alerts_repository_id
687 ON drift_alerts(repository_id);
688
689 GO
690
691 CREATE TRIGGER trg_users_updated_at
692 ON users AFTER UPDATE AS
693 BEGIN
694     SET NOCOUNT ON;
695     UPDATE u SET updated_at = GETDATE()
696     FROM users u INNER JOIN inserted i ON u.user_id = i.user_id
697     ;
698 END;
699 GO
700
701 CREATE TRIGGER trg_workspaces_updated_at
702 ON workspaces AFTER UPDATE AS
703 BEGIN
704     SET NOCOUNT ON;
705     UPDATE w SET updated_at = GETDATE()
706     FROM workspaces w INNER JOIN inserted i ON w.workspace_id =
707     i.workspace_id;
708 END;
709 GO
710
711 CREATE TRIGGER trg_documents_updated_at
712 ON documents AFTER UPDATE AS
713 BEGIN
714     SET NOCOUNT ON;
715     UPDATE d SET updated_at = GETDATE()
716     FROM documents d INNER JOIN inserted i ON d.document_id = i
717     .document_id;
718 END;
719 GO
720
721 CREATE TRIGGER trg_document_versions_updated_at
722 ON document_versions AFTER UPDATE AS
723 BEGIN
724     SET NOCOUNT ON;
725     UPDATE v SET updated_at = GETDATE()
726     FROM document_versions v INNER JOIN inserted i ON v.
727     version_id = i.version_id;
728 END;
729 GO
730
731 CREATE TABLE commit_files (
732     commit_id UNIQUEIDENTIFIER NOT NULL,
733     file_id    UNIQUEIDENTIFIER NOT NULL,

```

```

730
731     CONSTRAINT pk_commit_files PRIMARY KEY (commit_id, file_id)
732 ,
733     CONSTRAINT fk_cf_commit
734     FOREIGN KEY (commit_id) REFERENCES commits(commit_id)
735     ON DELETE CASCADE,
736     CONSTRAINT fk_cf_file
737     FOREIGN KEY (file_id) REFERENCES files(file_id) ON
738     DELETE CASCADE
739 );
740
741 CREATE TABLE grounding_evidence_code_entities (
742     evidence_id UNIQUEIDENTIFIER NOT NULL,
743     code_entity_id UNIQUEIDENTIFIER NOT NULL,
744
745     CONSTRAINT pk_ge_ce PRIMARY KEY (evidence_id,
746     code_entity_id),
747     CONSTRAINT fk_gece_evidence
748     FOREIGN KEY (evidence_id) REFERENCES
749     grounding_evidences(evidence_id) ON DELETE CASCADE,
750     CONSTRAINT fk_gece_entity
751     FOREIGN KEY (code_entity_id) REFERENCES code_entities(
752     code_entity_id) ON DELETE CASCADE
753 );
754
755 CREATE TABLE branch_commits (
756     branch_id UNIQUEIDENTIFIER NOT NULL,
757     commit_id UNIQUEIDENTIFIER NOT NULL,
758
759     CONSTRAINT pk_branch_commits PRIMARY KEY (branch_id,
760     commit_id),
761     CONSTRAINT fk_bc_branch
762     FOREIGN KEY (branch_id) REFERENCES branches(branch_id)
763     ON DELETE CASCADE,
764     CONSTRAINT fk_bc_commit
765     FOREIGN KEY (commit_id) REFERENCES commits(commit_id)
766     ON DELETE CASCADE
767 );
768
769 CREATE TABLE pull_request_commits (
770     pr_id UNIQUEIDENTIFIER NOT NULL,
771     commit_id UNIQUEIDENTIFIER NOT NULL,
772
773     CONSTRAINT pk_pr_commits PRIMARY KEY (pr_id, commit_id),
774     CONSTRAINT fk_prc_pr
775     FOREIGN KEY (pr_id) REFERENCES pull_requests(pr_id) ON
776     DELETE CASCADE,
777     CONSTRAINT fk_prc_commit

```

```

768         FOREIGN KEY (commit_id) REFERENCES commits(commit_id)
769         ON DELETE CASCADE
770 );
771 CREATE TABLE commit_parents (
772     commit_id          UNIQUEIDENTIFIER NOT NULL,
773     parent_commit_id UNIQUEIDENTIFIER NOT NULL,
774
775     CONSTRAINT pk_commit_parents PRIMARY KEY (commit_id,
776     parent_commit_id),
777     CONSTRAINT fk_cp_commit
778         FOREIGN KEY (commit_id) REFERENCES commits(commit_id),
779     CONSTRAINT fk_cp_parent
780         FOREIGN KEY (parent_commit_id) REFERENCES commits(
781     parent_commit_id),
782     CONSTRAINT chk_commit_parents_not_self
783         CHECK (commit_id <> parent_commit_id)
784 );
785 GO

```

Chương 5

Các Truy vấn Hệ thống Demo

Dưới đây là danh sách các truy vấn SQL minh họa cho các tính năng của hệ thống.

5.1 Truy vấn Phân quyền (RBAC): Danh sách Người dùng và Vai trò trong Workspace

Hiển thị nhân sự đang tham gia vào các không gian làm việc và chức danh của họ.

```
1 SELECT
2     w.name AS WorkspaceName,
3     u.first_name + ' ' + u.last_name AS FullName,
4     u.email,
5     r.role_name AS Role,
6     wm.status AS MemberStatus
7 FROM workspaces w
8 JOIN workspace_members wm ON w.workspace_id = wm.workspace_id
9 JOIN users u ON wm.user_id = u.user_id
10 JOIN roles r ON wm.role_id = r.role_id
11 ORDER BY w.name, r.role_name;
```

5.2 Truy vấn Phân quyền: Kiểm tra chi tiết Quyền hạn (Permissions) của một Vai trò

Demo cách hệ thống biết Developer hay Manager được phép làm gì.

```

1 SELECT
2     r.role_name,
3     p.permission_code,
4     p.description
5 FROM roles r
6 JOIN role_permissions rp ON r.role_id = rp.role_id
7 JOIN permissions p ON rp.permission_id = p.permission_id
8 WHERE r.role_name = 'developer'
9 ORDER BY p.permission_code;

```

5.3 Truy vấn Quản lý Repo: Thống kê số lượng Nhánh (Branch) theo từng Repository

Hiển thị tổng quan các kho lưu trữ mã nguồn.

```

1 SELECT
2     r.name AS RepositoryName,
3     r.default_branch,
4     COUNT(b.branch_id) AS TotalBranches
5 FROM repositories r
6 LEFT JOIN branches b ON r.repository_id = b.repository_id
7 GROUP BY r.name, r.default_branch;

```

5.4 Truy vấn Git Workflow: Chi tiết các Commit nằm trong một Pull Request

Sử dụng bảng trung gian pull_request_commits để truy xuất lịch sử đẩy code.

```

1 SELECT
2     pr.pr_number,
3     pr.title AS PR_Title,
4     c.commit_hash,
5     c.author_name,
6     c.message AS CommitMessage,
7     c.committed_at
8 FROM pull_requests pr
9 JOIN pull_request_commits prc ON pr.pr_id = prc.pr_id
10 JOIN commits c ON prc.commit_id = c.commit_id
11 WHERE pr.status = 'MERGED'
12 ORDER BY pr.pr_number, c.committed_at DESC;

```

5.5 Truy vấn Git Workflow: Phân tích Dòng code thay đổi (Code Diff)

Demo việc hệ thống lưu trữ sự khác biệt mã nguồn khi lập trình viên sửa file.

```
1 SELECT
2     c.commit_hash,
3     f.file_path,
4     cd.change_type,
5     cd.additions,
6     cd.deletions,
7     cd.diff_content
8 FROM commits c
9 JOIN code_diffs cd ON c.commit_id = cd.commit_id
10 JOIN files f ON cd.file_id = f.file_id
11 WHERE cd.change_type = 'MODIFIED';
```

5.6 Truy vấn Cây cú pháp (AST): Danh sách Thực thể mã nguồn (Code Entities)

Hiển thị các hàm, lớp, biến được hệ thống bóc tách từ file source code.

```
1 SELECT
2     f.file_path,
3     ce.entity_name,
4     ce.entity_type,
5     ce.signature,
6     ce.start_line,
7     ce.end_line
8 FROM files f
9 JOIN code_entities ce ON f.file_id = ce.file_id
10 ORDER BY f.file_path, ce.start_line;
```

5.7 Truy vấn Hệ quy AST: Sự phụ thuộc giữa các đoạn code (Dependencies)

Tìm xem hàm A đang gọi đến hoặc phụ thuộc vào hàm B nào.

```
1 SELECT
2     src.entity_name AS SourceEntity,
3     src.entity_type AS SourceType,
4     d.dependency_type,
5     tgt.entity_name AS TargetEntity,
```

```

6      tgt.entity_type AS TargetType
7 FROM dependencies d
8 JOIN code_entities src ON d.source_entity_id = src.
   code_entity_id
9 JOIN code_entities tgt ON d.target_entity_id = tgt.
   code_entity_id
10 WHERE d.dependency_type = 'REFERENCES';

```

5.8 Truy vấn Sinh tài liệu: Cấu trúc của một Biểu mẫu (Template)

Demo các phân đoạn và điểm giữ chỗ (Placeholder) được định nghĩa bởi Manager.

```

1 SELECT
2     t.name AS TemplateName,
3     ts.order_index,
4     ts.heading_title,
5     p.variable_name,
6     p.expected_data_type
7 FROM templates t
8 JOIN template_sections ts ON t.template_id = ts.template_id
9 LEFT JOIN placeholders p ON ts.section_id = p.section_id
10 ORDER BY t.name, ts.order_index;

```

5.9 Truy vấn Automation: Cách Placeholder map với Source Code

Chứng minh tính tự động hóa: Điểm điền tài liệu nào đang map với hàm code nào.

```

1 SELECT
2     p.variable_name AS Placeholder,
3     tm.extraction_rule,
4     ce.entity_name AS MappedCodeEntity,
5     ce.signature AS CodeSignature
6 FROM template_mappings tm
7 JOIN placeholders p ON tm.placeholder_id = p.placeholder_id
8 JOIN code_entities ce ON tm.code_entity_id = ce.code_entity_id
9 WHERE tm.is_active = 1;

```

5.10 Truy vấn Vòng đời Tài liệu: Các Phiên bản của một Tài liệu

Xem lịch sử cập nhật version của một file tài liệu.

```
1 SELECT
2     d.title AS DocumentTitle,
3     d.document_type,
4     dv.version_number,
5     dv.status AS VersionStatus,
6     dv.created_at
7 FROM documents d
8 JOIN document_versions dv ON d.document_id = dv.document_id
9 ORDER BY d.title, dv.created_at DESC;
```

5.11 Truy vấn Review: Đọc các Comment góp ý trên Tài liệu

Kiểm tra xem Technical Lead hoặc Manager đã nhận xét gì về tài liệu.

```
1 SELECT
2     dv.version_number,
3     u.first_name + ' ' + u.last_name AS Reviewer,
4     r.decision AS ReviewDecision,
5     rc.content AS CommentContent,
6     rc.is_resolved
7 FROM document_versions dv
8 JOIN reviews r ON dv.version_id = r.version_id
9 JOIN review_comments rc ON r.review_id = rc.review_id
10 JOIN users u ON rc.user_id = u.user_id;
```

5.12 Truy vấn Workflow Phê duyệt: Lịch sử Approval

Truy xuất ai đã chốt duyệt phiên bản tài liệu cuối cùng.

```
1 SELECT
2     dv.version_number,
3     u.first_name + ' ' + u.last_name AS Approver,
4     a.decision,
5     a.rejection_reason,
6     a.approved_at
7 FROM document_versions dv
8 JOIN approvals a ON dv.version_id = a.version_id
9 JOIN users u ON a.user_id = u.user_id;
```

5.13 Truy vấn Grounding: Dẫn chứng mã nguồn của Tài liệu

Chứng minh tài liệu không bịa đặt (hallucination) nhờ bằng chứng gốc.

```
1 SELECT
2     dv.version_number ,
3     ge.purpose ,
4     ge.snippet_content ,
5     ce.entity_name AS SourceCodeEntity ,
6     f.file_path
7 FROM document_versions dv
8 JOIN grounding_evidences ge ON dv.version_id = ge.version_id
9 LEFT JOIN grounding_evidence_code_entities gece ON ge.
10     evidence_id = gece.evidence_id
11 LEFT JOIN code_entities ce ON gece.code_entity_id = ce.
12     code_entity_id
13 LEFT JOIN files f ON ce.file_id = f.file_id;
```

5.14 Truy vấn Cảnh báo Trôi dạt (Drift Alert): Các sai lệch chưa xử lý

Hiển thị các cảnh báo khi code mới sửa làm tài liệu cũ bị sai lệch.

```
1 SELECT
2     r.name AS Repository ,
3     da.drift_type ,
4     da.status AS AlertStatus ,
5     da.detected_at ,
6     ge.snippet_content AS OldDocumentedCode
7 FROM drift_alerts da
8 JOIN repositories r ON da.repository_id = r.repository_id
9 JOIN grounding_evidences ge ON da.evidence_id = ge.evidence_id
10 WHERE da.status = 'OPEN'
11 ORDER BY da.detected_at DESC;
```

5.15 Truy vấn Audit: Nhật ký thay đổi hệ thống (Change Log)

Truy xuất toàn bộ lịch sử biến động để audit hệ thống.

```
1 SELECT
2     dv.version_number ,
3     cl.change_description ,
```

```
4     cl.diff_content,  
5     cl.created_at  
6 FROM change_logs cl  
7 JOIN document_versions dv ON cl.version_id = dv.version_id  
8 ORDER BY cl.created_at DESC;
```


Phần IV

Tổng Kết

Chương 6

Tổng Kết

Đề tài đã hoàn thành xuất sắc quy trình thiết kế cơ sở dữ liệu toàn diện cho hệ thống **LivingDocs** trải dài qua ba giai đoạn chuẩn hóa từ Stage 1 đến Stage 3. Ở Mức quan niệm (Stage 1), mô hình ERD và UML Class Diagram đã chuẩn hóa thành công các thực thể nghiệp vụ phức tạp như phân quyền Scoped RBAC, luồng kiểm duyệt 2 cấp và cơ chế lưu minh chứng AI. Chuyển sang Mức logic (Stage 2), hệ thống được ánh xạ chính xác sang Lược đồ quan hệ và chứng minh đạt chuẩn 3NF, giúp loại bỏ triệt để sự dư thừa dữ liệu cũng như các bất thường khi thêm, sửa, xóa. Tại Mức vật lý (Stage 3), các bảng được tối ưu hóa kiểu dữ liệu, hệ thống chỉ mục (Index) và ràng buộc toàn vẹn nhằm nâng cao hiệu năng truy vấn ở mức tối đa. Kết quả thu được tạo nên một nền tảng kiến trúc dữ liệu vững chắc, sẵn sàng đáp ứng khả năng mở rộng và vận hành ổn định cho dự án khi triển khai thực tế. Đây là minh chứng rõ nét cho tính hiệu quả của việc áp dụng quy trình thiết kế CSDL bài bản vào việc xây dựng một hệ thống phần mềm chuyên nghiệp.